

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH — TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN

IT003: CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

ULTIMATE HANDBOOK & EXAM REVISION GUIDE

Tài liệu Giảng dạy, Cẩm nang Tra cứu & Bộ Đề Thi Mẫu Chuẩn UIT

Biên soạn: Võ Trọng Phúc

MỤC LỤC TỔNG QUAN (TABLE OF CONTENTS)

1. **PART 0:** Cách học và Chinh phục môn IT003 (Roadmap & Progression)
2. **PART I:** Phân tích Thuật toán & Độ phức tạp (Big-O, Loop Analysis)
3. **PART II:** Thuật toán Tìm kiếm (Linear & Binary Search)
4. **PART III:** Các Thuật toán Sắp xếp (10 Sorting Algorithms & Complexity Matrix)
5. **PART IV:** Danh sách Liên kết (Singly & Doubly Linked List)
6. **PART V:** Ngăn xếp & Hàng đợi (Stack, Queue & Circular Queue)
7. **PART VI:** Cây & Cây Nhị phân (Trees & Binary Tree Traversals)
8. **PART VII:** Cây Nhị phân Tìm kiếm (BST Search, Insert & Deletion)
9. **PART VIII:** Cây Cân bằng AVL (Balance Factors & 4 Rotations Cheat Sheet)
10. **PART IX:** Heaps & Hàng đợi Ưu tiên (Max/Min Heap & Array Indexing)
11. **PART X:** Cây B-Tree (Order 5 B-Tree Split & Promotion Rules)
12. **PART XI:** Bảng Băm (Hash Table, Probing & Unsuccessful Search Comparisons)
13. **PART XII:** Đồ thị & Biểu diễn (Adjacency Matrix vs Adjacency List)
14. **PART XIII:** Duyệt Đồ thị (BFS with Queue & DFS with Stack/Recursion)
15. **PART XIV:** Đường đi Ngắn nhất (Dijkstra Algorithm & Tracing Table)
16. **PART XV:** Exam Toolkit & Bộ Đề Ôn Thi Mẫu Chuẩn IT003

LÀM SAO ĐỂ CHINH PHỤC IT003? (HOW TO MASTER IT003)

1. Intuition (Trực giác & Tại sao phải học IT003?)

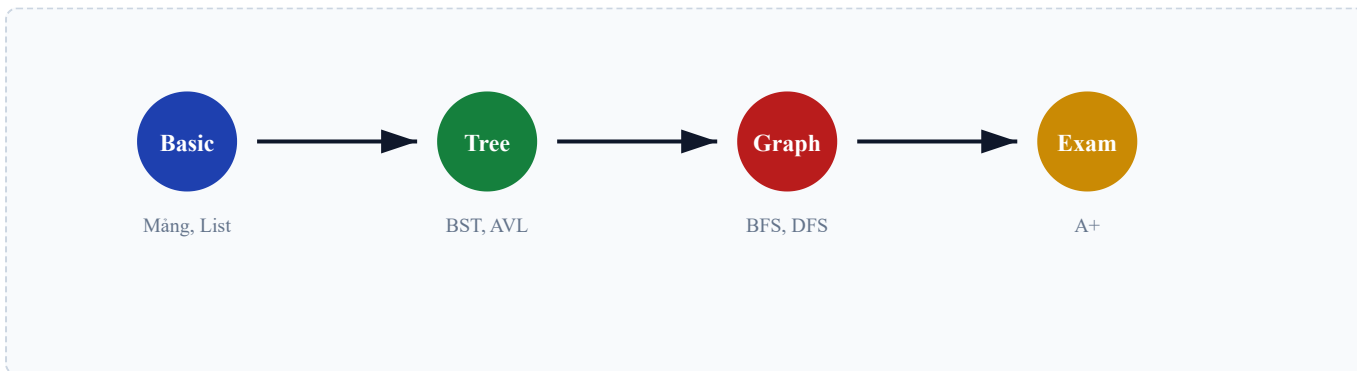
IT003 (Cấu trúc Dữ liệu và Giải thuật) không chỉ là một môn học bắt buộc để ra trường, mà là "ngôn ngữ chung" của mọi kỹ sư phần mềm. Bạn không thể xây dựng một hệ thống chịu tải hàng triệu người dùng nếu không biết phân biệt $\mathcal{O}(n)$ và $\mathcal{O}(\log n)$. Môn học này dạy bạn cách tổ chức dữ liệu một cách thông minh để giải quyết vấn đề hiệu quả nhất.

2. Formal Model (Mô hình Syllabus IT003)

Syllabus IT003 tại UIT được chia thành 3 trụ cột chính:

- Trụ cột 1 (Tuyến tính & Sắp xếp):** Mảng, Danh sách liên kết, Stack, Queue, 10 Thuật toán Sắp xếp, Tìm kiếm nhị phân.
- Trụ cột 2 (Cây & Phân cấp):** Cây nhị phân, BST, AVL, B-Tree, Heaps.
- Trụ cột 3 (Băm & Đồ thị):** Hash Table, Đồ thị, BFS, DFS, Dijkstra.

3. Visual Diagram (Lộ trình chinh phục)



4. Algorithm & Mechanics (Kỹ năng Dry-run)

Kỹ năng quan trọng nhất để qua môn IT003 là **Dry-run** (Trace code bằng tay). Đừng vội gõ code vào IDE. Hãy lấy một tờ giấy, vẽ các ô vuông tượng trưng cho biến và mảng, rồi dùng bút chì cập nhật giá trị từng bước một khi vòng lặp chạy.

5. C++ Code (Thiết lập môi trường)

```
#include <iostream>
using namespace std;
```

```
int main() {  
    cout << "Welcome to IT003 DSA UIT!" << endl;  
    return 0;  
}
```

6. Complexity Analysis (Tư duy tối ưu)

Không chỉ giải quyết bài toán, bạn phải tự hỏi: "Mình có thể làm nó nhanh hơn không?". Giảm từ $\mathcal{O}(n^2)$ xuống $\mathcal{O}(n \log n)$ là sự khác biệt giữa 1 giây và 10 tiếng đồng hồ khi xử lý dữ liệu lớn.

7. Dry Run Table (Mẫu theo dõi biến)

Vòng lặp	Biến i	Biến j	Trạng thái dữ liệu
Khởi tạo	0	-	[3, 1, 4]
Lần 1	0	1	So sánh a[0] và a[1]

8. Common Errors (Những sai lầm thường gặp)

- Học thuộc code thay vì hiểu bản chất thuật toán.
- Chỉ đọc slide mà không lấy giấy nháp ra vẽ sơ đồ.
- Không xử lý các trường hợp biên (Null pointer, mảng rỗng).

9. Exam Connection & Practice

IT003 EXAM STYLE

Đề thi IT003 tập trung rất mạnh vào khả năng Dry-run. Một tỷ lệ đáng kể câu hỏi trong đề thi yêu cầu bạn chạy tay thuật toán từng bước trên một cây, một đồ thị, hoặc một mảng để ra kết quả cuối cùng.

Tóm kết Chương 0

Bí quyết: Đọc hiểu Intuition -> Vẽ sơ đồ -> Trace tay -> Hiểu Complexity. Cùng bắt đầu hành trình nhé!

PHÂN TÍCH THUẬT TOÁN & ĐỘ PHỨC TẠP (ALGORITHM ANALYSIS)

1. Intuition (Trực giác & Động lực)

Làm sao để biết thuật toán A chạy nhanh hơn thuật toán B mà không cần phải code và chạy thử trên cùng một máy tính? Đo bằng giây (seconds) là không đáng tin cậy vì phụ thuộc vào phần cứng. Do đó, chúng ta đo lường hiệu năng bằng cách đếm số lượng "phép toán cơ bản" dựa trên kích thước dữ liệu đầu vào n . Đó là khởi nguồn của Ký pháp Big-O.

2. Formal Model (Định nghĩa Big-O, Big-Omega, Big-Theta)

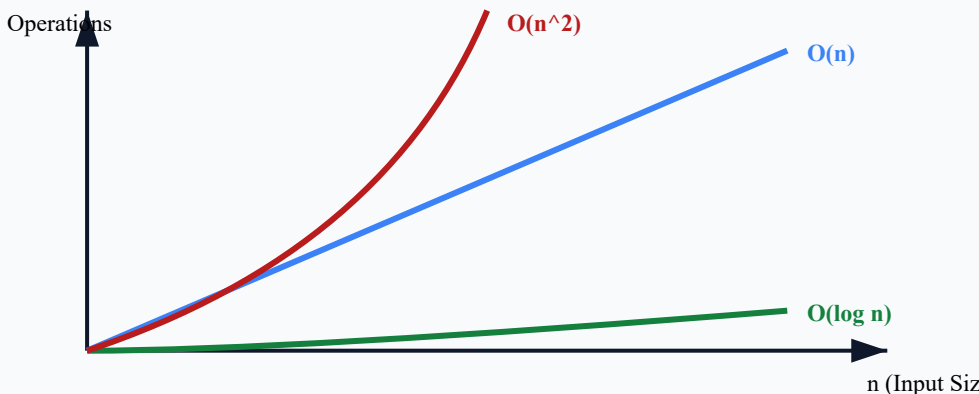
- $\mathcal{O}(f(n))$ (Big-O): Cận trên tiệm cận (Upper Bound). Tốc độ tăng trưởng không vượt quá $f(n)$.
- $\Omega(f(n))$ (Big-Omega): Cận dưới tiệm cận (Lower Bound). Tốc độ tăng trưởng tối thiểu đạt $f(n)$.
- $\Theta(f(n))$ (Big-Theta): Cận chặt tiệm cận (Tight Bound). Tốc độ tăng trưởng vừa đúng bằng bậc $f(n)$ (khi cận trên và cận dưới trùng nhau).

Thứ tự độ phức tạp từ Tốt đến Xấu: $\mathcal{O}(1) < \mathcal{O}(\log n) < \mathcal{O}(n) < \mathcal{O}(n \log n) < \mathcal{O}(n^2) < \mathcal{O}(2^n) < \mathcal{O}(n!)$

IT003 PRECISION NOTE

Phân biệt Ký pháp Tiệm cận và Kích bản Đầu vào: Big-O, Big-Omega, Big-Theta mô tả *cận toán học* của hàm số, không đồng nhất với *kịch bản dữ liệu đầu vào* (Best/Worst/Average Case). Ví dụ: Worst-case của Insertion Sort có độ phức tạp tiệm cận chặt là $\Theta(n^2)$, trong khi Best-case của nó có độ phức tạp tiệm cận chặt là $\Theta(n)$.

3. Visual Diagram (Đồ thị Tăng trưởng Hàm số)



4. Algorithm & Mechanics (Phân tích Vòng lặp)

Quy tắc cộng: Hai khối lệnh nối tiếp nhau $\mathcal{O}(f(n)) + \mathcal{O}(g(n)) = \mathcal{O}(\max(f(n), g(n)))$.

Quy tắc nhân: Vòng lặp lồng nhau $\mathcal{O}(f(n)) \times \mathcal{O}(g(n)) = \mathcal{O}(f(n) \cdot g(n))$.

5. C++ Educational Implementation (Phân tích Code thực tế)

```
// Ví dụ 1: O(n)
for (int i = 0; i < n; i++) {
    cout << i;
}

// Ví dụ 2: O(n^2)
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        cout << i + j;
    }
}

// Ví dụ 3: O(log n)
int k = n;
while (k > 1) {
    k = k / 2;
}
```

6. Complexity Analysis (Phân tích Độ quy)

Với hàm đệ quy, độ phức tạp không chỉ đếm số lệnh mà còn phụ thuộc vào **Độ sâu của cây đệ quy (Recursion Tree)**. Ví dụ: Tính Fibonacci đệ quy $F(n) = F(n-1) + F(n-2)$ sinh ra một cây nhị phân có chiều cao n , dẫn đến độ phức tạp thời gian $\mathcal{O}(2^n)$ vô cùng chậm.

7. Dry Run Table (Trace Vòng Lặp Phức Tạp)

Trace code: `for (int i = 1; i < n; i = i * 2)` với $n = 10$:

Vòng lặp	Giá trị i	Biểu diễn toán học
1	1	2^0
2	2	2^1
3	4	2^2
4	8	2^3
Kết thúc	16 (Dừng vì $16 \geq 10$)	$2^k \geq n \Rightarrow k = \lceil \log_2 n \rceil$

8. Common Errors & Pitfalls

- Bỏ qua hằng số quá sớm: $\mathcal{O}(100n)$ và $\mathcal{O}(n)$ là giống nhau về lý thuyết, nhưng trong thực tế $100n$ chậm hơn nhiều.
- Nhầm lẫn giữa Tốt nhất và Xấu nhất: Tìm kiếm tuyến tính có Best-case là $\Omega(1)$ nếu phần tử ở ngay đầu, nhưng Worst-case là $\mathcal{O}(n)$. Đừng nhầm lẫn!

9. Quick Recall & Exam Style

IT003 EXAM STYLE

Bẫy Đề Thi: Khi tính độ phức tạp của hàm đệ quy, sinh viên thường quên tính độ phức tạp Không gian (Space Complexity) của Call Stack. Mọi lệnh gọi đệ quy đều tốn $\mathcal{O}(1)$ bộ nhớ cho mỗi stack frame. Do đó đệ quy sâu n tầng sẽ tốn $\mathcal{O}(n)$ bộ nhớ.

- **Level 0:** Hàm $f(n) = 3n^2 + 5n + 10$ có Big-O là gì?
Đáp án: $\mathcal{O}(n^2)$ (Bỏ hằng số, lấy bậc cao nhất).
- **Level 1:** Vòng lặp `for (int i = 0; i < n; i += 2)` có độ phức tạp là bao nhiêu?
Đáp án: Chạy $n/2$ lần $\Rightarrow \mathcal{O}(n)$.

Tóm kết Chương 1

Big-O là lăng kính để bạn nhìn ra thuật toán nào sẽ trụ vững khi dữ liệu phình to đến hàng triệu phần tử.

PART II

THUẬT TOÁN TÌM KIẾM (SEARCHING ALGORITHMS)

1. Intuition (Trực giác & Động lực)

Tìm kiếm là thao tác cơ bản nhất trong khoa học máy tính. Hãy tưởng tượng việc tìm một từ trong từ điển 1,000 trang: bạn sẽ lật từng trang từ đầu (Linear Search) hay mở đôi cuốn sách ở giữa rồi tiếp tục chia đôi (Binary Search)? Bài toán tìm kiếm quyết định hiệu năng của mọi cơ sở dữ liệu lớn.

2. Formal Model (Định nghĩa Thuật toán)

Cho dãy $A = (a_0, a_1, \dots, a_{n-1})$ và giá trị x . Tìm vị trí index $i \in [0, n - 1]$ sao cho $a_i = x$, hoặc trả về -1 nếu $x \notin A$.

IT003 CONVENTION : Với Tìm kiếm Nhị phân, điều kiện tiên quyết bắt buộc là dãy A phải được sắp xếp theo một thứ tự xác định (tăng dần hoặc giảm dần).

3. Visual Diagram (Sơ đồ Tìm kiếm Nhị phân)

3	7	11	15	19	24	31
0	1 (L)	2	3 (MID)	4	5 (R)	6

4. Algorithm & Mechanics (Cơ chế hoạt động)

So sánh x với phần tử ở giữa $mid = left + (right - left) / 2$. Nếu $x == a[mid]$, trả về mid . Nếu $x < a[mid]$, thu hẹp phạm vi về bên trái ($right = mid - 1$). Ngược lại về bên phải ($left = mid + 1$).

5. C++ Educational & STL Code

```
// Educational Iterative Binary Search
int binarySearch(int a[], int n, int x) {
    int left = 0, right = n - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2; // Phòng tránh tràn số int
        if (a[mid] == x) return mid;
        if (a[mid] < x) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}

// STL Implementation Example
#include <iostream>
#include <vector>
#include <algorithm>

bool searchSTL(const std::vector<int>& v, int x) {
    return std::binary_search(v.begin(), v.end(), x);
}
```

6. Complexity Analysis & Invariants

- **Linear Search:** Best Case $\mathcal{O}(1)$, Worst Case $\Theta(n)$, Average Case $\Theta(n)$. Space: $\mathcal{O}(1)$.
- **Binary Search:** Best Case $\mathcal{O}(1)$, Worst Case $\Theta(\log_2 n)$, Average Case $\Theta(\log_2 n)$. Space: $\mathcal{O}(1)$ với lặp, $\mathcal{O}(\log n)$ với đệ quy.

7. Dry Run Table (Chạy tay chi tiết)

Tìm $x = 19$ trong mảng $A = [3, 7, 11, 15, 19, 24, 31]$ ($n = 7$):

Bước	left	right	mid	a[mid]	So sánh $x=19$	Hành động
1	0	6	3	15	$15 < 19$	<code>left = mid + 1 = 4</code>
2	4	6	5	24	$24 > 19$	<code>right = mid - 1 = 4</code>
3	4	4	4	19	$19 == 19$	Tìm thấy! Trả về index 4

8. Common Errors & IT003 Exam Style

IT003 EXAM STYLE

Lỗi thường gặp: Tính `mid = (left + right) / 2` có thể bị tràn số khi `left + right > INT_MAX`. Luôn dùng `left + (right - left) / 2`. Ngoài ra, điều kiện vòng lặp phải là `left ≤ right` (dấu `≤` bắt buộc để không bỏ sót trường hợp 1 phần tử).

9. Quick Recall & Exercise Ladder

- Level 0:** Tìm kiếm Nhị phân có thể áp dụng cho danh sách liên kết đơn được sắp xếp không? Vì sao?
Đáp án: Không hiệu quả, vì DSLK không hỗ trợ truy xuất ngẫu nhiên $a[mid]$ trong $\mathcal{O}(1)$, làm độ phức tạp tăng lên $\mathcal{O}(n)$.
- Level 1:** Cho mảng $A = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]$. Trình bày bảng trace tìm kiếm $x = 23$.
Gợi ý: Bước 1: $mid=4$ ($16 < 23$) $\Rightarrow$ Bước 2: $mid=7$ ($56 > 23$) $\Rightarrow$ Bước 3: $mid=5$ ($23 == 23$).

Tóm kết Chương 2

Tìm kiếm nhị phân giảm phạm vi tìm kiếm đi một nửa sau mỗi bước ($\mathcal{O}(\log n)$). Hãy luôn nhớ kiểm tra mảng đã sắp xếp trước khi gọi thuật toán!

CÁC THUẬT TOÁN SẮP XẾP (SORTING ALGORITHMS)

1. Intuition (Trực giác & Động lực)

Sắp xếp dữ liệu giúp việc tìm kiếm, thống kê và xử lý trở nên dễ dàng và nhanh chóng hơn. Trong thực tế, các thuật toán sắp xếp được chia làm 2 nhóm chính: **Sắp xếp cơ bản** ($O(n^2)$) dễ cài đặt và **Sắp xếp nâng cao** ($O(n \log n)$) cho dữ liệu lớn.

2. Formal Model & Complexity Matrix

 Bảng Tổng Hợp 10 Thuật Toán Sắp Xếp Chuẩn IT003

Thuật toán	Best Case	Average Case	Worst Case	Space	Stable?	In-Place?
Selection Sort	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$\mathcal{O}(1)$	No	Yes
Interchange Sort	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$\mathcal{O}(1)$	No	Yes
Bubble Sort (Basic)	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$\mathcal{O}(1)$	Yes	Yes
Bubble Sort (Optimized)	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$\mathcal{O}(1)$	Yes	Yes
Insertion Sort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$\mathcal{O}(1)$	Yes	Yes
Binary Insertion	$\Theta(n \log n)^*$	$\Theta(n^2)$	$\Theta(n^2)$	$\mathcal{O}(1)$	Yes	Yes
Shell Sort	Phụ thuộc gap*	Phụ thuộc gap*	Phụ thuộc gap*	$\mathcal{O}(1)$	No	Yes
Heap Sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\mathcal{O}(1)$	No	Yes
Quick Sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$	$\mathcal{O}(\log n)$	No	Yes
Merge Sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$	Yes	No
Radix Sort	$\Theta(d \cdot n)$	$\Theta(d \cdot n)$	$\Theta(d \cdot n)$	$\Theta(n + k)^*$	Yes	No

* Ghi chú Học thuật:

- *Interchange Sort*: Thuật toán này không ổn định (NOT STABLE) do việc đổi chỗ các phần tử ở xa nhau có thể làm thay đổi thứ tự tương đối của các phần tử bằng nhau.
- *Binary Insertion*: Với implementation bên dưới luôn chạy Binary Search ở mỗi vòng, tổng số phép so sánh ở input đã sắp xếp là $\Theta(n \log n)$, số lần dời phần tử là 0, nên tổng thời gian Best Case của chính implementation này là $\Theta(n \log n)$. Không suy rộng kết quả này cho mọi biến thể Binary Insertion Sort.
- *Shell Sort*: Best/Average/Worst phụ thuộc mạnh vào chuỗi khoảng cách (gap sequence), không có một cận trung bình duy nhất áp dụng cho mọi dãy gap. Với dãy Shell gốc $(n/2, n/4, \dots)$, Worst Case là $\Theta(n^2)$.
- *Radix Sort*: k là cơ số của hệ đếm (ví dụ $k = 10$ với hệ thập phân).

3. Visual Diagram (Sơ đồ Phân hoạch Quick Sort)



4. The 10 Sorting Algorithms

4.1. Selection Sort

Intuition: Tìm phần tử nhỏ nhất trong mảng chưa sắp xếp và đổi chỗ với phần tử đầu tiên của mảng đó.

```
void selectionSort(int a[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int min_idx = i;
        for (int j = i + 1; j < n; j++)
            if (a[j] < a[min_idx]) min_idx = j;
        std::swap(a[i], a[min_idx]);
    }
}
```

Complexity: $\Theta(n^2)$ cho mọi trường hợp. **Stability:** No.

Bước i	Mảng	Thao tác
0	[5, 2, 4, 1]	min=1, swap(5,1) → [1, 2, 4, 5]

4.2. Interchange Sort

Intuition: So sánh cặp (i, j) , nếu sai thứ tự thì đổi chỗ ngay lập tức.

```
void interchangeSort(int a[], int n) {
    for (int i = 0; i < n - 1; i++)
        for (int j = i + 1; j < n; j++)
            if (a[i] > a[j]) std::swap(a[i], a[j]);
}
```

Complexity: $\Theta(n^2)$. Stability: No.

Bước i	Mảng	Thao tác
0	[3, 2, 1]	swap(3,2) $\rightarrow$ [2,3,1], swap(2,1) $\rightarrow$ [1,3,2]

4.3. Bubble Sort

Intuition: Nổi bọt, đẩy phần tử lớn nhất xuống cuối mảng qua từng lượt.

```
void bubbleSort(int a[], int n) {
    for (int i = 0; i < n - 1; i++) {
        bool swapped = false;
        for (int j = 0; j < n - i - 1; j++)
            if (a[j] > a[j+1]) { std::swap(a[j], a[j+1]); swapped = true; }
        if (!swapped) break;
    }
}
```

Complexity: Best $\Theta(n)$, Worst $\Theta(n^2)$. Stability: Yes.

Bước i	Mảng	Thao tác
0	[3, 2, 1]	swap(3,2), swap(3,1) $\rightarrow$ [2, 1, 3]

4.4. Insertion Sort

Intuition: Chèn từng phần tử vào vị trí thích hợp trong mảng đã sắp xếp ở phía trước.

```
void insertionSort(int a[], int n) {
    for (int i = 1; i < n; i++) {
        int key = a[i], j = i - 1;
        while (j  $\geq$  0 && a[j] > key) { a[j + 1] = a[j]; j--; }
        a[j + 1] = key;
    }
}
```

Complexity: Best $\Theta(n)$, Worst $\Theta(n^2)$. Stability: Yes.

Bước i	Mảng	Thao tác
1	[3, 2, 1]	key=2, dời 3 $\rightarrow$ [2, 3, 1]

4.5. Binary Insertion Sort

Intuition: Cải tiến Insertion Sort bằng cách dùng tìm kiếm nhị phân để tìm vị trí chèn.

```
void binaryInsertionSort(int a[], int n) {
    for (int i = 1; i < n; i++) {
        int key = a[i], left = 0, right = i - 1;
        while (left ≤ right) {
            int mid = (left + right) / 2;
            if (a[mid] > key) right = mid - 1;
            else left = mid + 1;
        }
        for (int j = i - 1; j ≥ left; j--) a[j + 1] = a[j];
        a[left] = key;
    }
}
```

Complexity: $\Theta(n^2)$. **Stability:** Yes.

Bước i	Mảng	Thao tác
1	[3, 2, 1]	key=2, BS tìm pos=0, dời 3 → [2, 3, 1]

4.6. Shell Sort

Intuition: Sắp xếp các phần tử cách nhau một khoảng gap, sau đó giảm dần gap.

```
void shellSort(int a[], int n) {
    for (int gap = n/2; gap > 0; gap /= 2) {
        for (int i = gap; i < n; i++) {
            int temp = a[i], j;
            for (j = i; j ≥ gap && a[j - gap] > temp; j -= gap)
                a[j] = a[j - gap];
            a[j] = temp;
        }
    }
}
```

Complexity: Phụ thuộc gap. **Stability:** No.

Gap	Mảng	Thao tác
2	[4, 3, 2, 1]	→ [2, 1, 4, 3]

4.7. Heap Sort

Intuition: Xây dựng Max-Heap rồi liên tục lấy phần tử lớn nhất đẩy về cuối.

```
void heapify(int a[], int n, int i) {
    int largest = i, l = 2*i + 1, r = 2*i + 2;
```

```

    if (l < n && a[l] > a[largest]) largest = l;
    if (r < n && a[r] > a[largest]) largest = r;
    if (largest != i) { std::swap(a[i], a[largest]); heapify(a, n, largest); }
}
void heapSort(int a[], int n) {
    for (int i = n / 2 - 1; i ≥ 0; i--) heapify(a, n, i);
    for (int i = n - 1; i > 0; i--) { std::swap(a[0], a[i]); heapify(a, i, 0); }
}

```

Complexity: $\Theta(n \log n)$. **Stability:** No.

Phase	Mảng	Thao tác
Build	[2, 1, 3]	→ [3, 1, 2]

4.8. Merge Sort

Intuition: Chia để trị, chia mảng thành 2 nửa, sắp xếp rồi trộn lại.

```

// Pseudocode Helper: void merge(int a[], int l, int m, int r);
void mergeSort(int a[], int l, int r) {
    if (l < r) {
        int m = l + (r - l) / 2;
        mergeSort(a, l, m); mergeSort(a, m + 1, r);
        merge(a, l, m, r); // Hàm trộn
    }
}

```

Complexity: $\Theta(n \log n)$. **Stability:** Yes.

Bước	Mảng	Thao tác
Trộn	[2] và [1]	→ [1, 2]

4.9. Quick Sort

Intuition: Chọn pivot, phân hoạch mảng thành phần nhỏ hơn và phần lớn hơn pivot.

```

// Pseudocode Helper: int partitionLomuto(int a[], int low, int high);
void quickSort(int a[], int low, int high) {
    if (low < high) {
        int pi = partitionLomuto(a, low, high);
        quickSort(a, low, pi - 1);
        quickSort(a, pi + 1, high);
    }
}

```

Complexity: Avg $\Theta(n \log n)$, Worst $\Theta(n^2)$. **Stability:** No.

Pivot	Mảng	Thao tác
30	[25, 40, 8, 30]	→ [25, 8, 30, 40]

4.10. Radix Sort

Intuition: Sắp xếp theo từng chữ số từ hàng đơn vị lên.

```
// Pseudocode Helper: int getMax(int a[], int n);
// Pseudocode Helper: void countSort(int a[], int n, int exp);
void radixSort(int a[], int n) {
    int max_val = getMax(a, n);
    for (int exp = 1; max_val / exp > 0; exp *= 10)
        countSort(a, n, exp); // Sắp xếp ổn định
}
```

Complexity: $\Theta(d \cdot n)$. Stability: Yes.

Exp	Mảng	Thao tác
1	[12, 10]	→ [10, 12]

8. Common Errors & IT003 Exam Style

IT003 EXAM STYLE

Bắt Đề Thi: Phân biệt Tính Ổn định (Stability) và Tính Tại chỗ (In-place). Quick Sort là **In-place** nhưng **Không ổn định**. Merge Sort **Ổn định** nhưng **Không In-place** ($\mathcal{O}(n)$ bộ nhớ phụ).

9. Quick Recall & Exercise Ladder

- Level 0:** Thuật toán nào có số phép so sánh $C = \frac{n(n-1)}{2}$ cố định trong mọi trường hợp?
Đáp án: Selection Sort và Interchange Sort.
- Level 1:** Thực hiện chạy tay 2 lượt đầu tiên của Bubble Sort trên mảng $A = [5, 1, 4, 2, 8]$.
Gợi ý: Lượt 1: [1, 4, 2, 5, 8]. Lượt 2: [1, 2, 4, 5, 8].

 **Tóm kết Chương 3**

Nắm vững bảng ma trận độ phức tạp 10 thuật toán sắp xếp và kỹ năng chạy tay từng lượt để đạt điểm tối đa câu 1 trong đề thi IT003!

DANH SÁCH LIÊN KẾT (LINKED LIST)

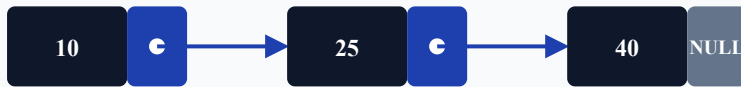
1. Intuition (Trực giác & Động lực)

Mảng tĩnh có kích thước cố định và thao tác chèn/xóa ở giữa tốn $\mathcal{O}(n)$ do phải dời hàng loạt phần tử. Danh sách liên kết (Linked List) giải quyết triệt để vấn đề này bằng cách cấp phát động từng nút (Node) nằm rải rác trên bộ nhớ Heap và liên kết chúng bằng các con trỏ (Pointer).

2. Formal Model (Định nghĩa Cấu trúc)

Một **Singly Linked List** gồm tập hợp các nút **Node**, trong đó mỗi nút chứa 2 thành phần: **data** (giá trị lưu trữ) và **pNext** (con trỏ chỉ tới nút kế tiếp). Danh sách được quản lý bởi con trỏ **pHead** (chỉ nút đầu) và **pTail** (chỉ nút cuối).

3. Visual Diagram (Sơ đồ Bộ nhớ Danh sách Liên kết)



4. Algorithm & Mechanics (Các Thao tác Con trỏ)

- **Add Head:** Tạo **pNew**. Cho **pNew→pNext = pHead**. Cập nhật **pHead = pNew**.
- **Add Tail:** Tạo **pNew**. Cho **pTail→pNext = pNew**. Cập nhật **pTail = pNew**.
- **Delete Head:** Lưu **temp = pHead**. Cho **pHead = pHead→pNext**. Giải phóng **delete temp**.
- **Search:** Duyệt từ đầu, nếu **p→data == val** thì trả về **p**.
- **Insert After:** Tìm nút **q**. Tạo **pNew**, **pNew→pNext = q→pNext**, **q→pNext = pNew**.
- **Count:** Duyệt qua danh sách và tăng biến đếm.

5. C++ Educational & STL Code

5.1 Singly Linked List

```
struct Node {
    int data;
    Node* pNext;
};
```

```

struct LinkedList {
    Node* pHead;
    Node* pTail;
};

void addHead(LinkedList& list, int val) {
    Node* p = new Node{val, nullptr};
    if (!list.pHead) list.pHead = list.pTail = p;
    else { p->pNext = list.pHead; list.pHead = p; }
}

void deleteHead(LinkedList& list) {
    if (!list.pHead) return;
    Node* temp = list.pHead;
    list.pHead = list.pHead->pNext;
    if (!list.pHead) list.pTail = nullptr;
    delete temp;
}

Node* search(LinkedList list, int val) {
    for (Node* p = list.pHead; p != nullptr; p = p->pNext)
        if (p->data == val) return p;
    return nullptr;
}

void insertAfter(LinkedList& list, Node* q, int val) {
    if (!q) return;
    Node* p = new Node{val, q->pNext};
    q->pNext = p;
    if (list.pTail == q) list.pTail = p;
}

int countNodes(LinkedList list) {
    int count = 0;
    for (Node* p = list.pHead; p != nullptr; p = p->pNext) count++;
    return count;
}

```

5.2 Doubly Linked List

```

struct DNode {
    int data;
    DNode* pPrev;
    DNode* pNext;
};

struct DLinkedList {
    DNode* pHead;
    DNode* pTail;
};

// Tradeoffs: Có thể duyệt 2 chiều dễ dàng, nhưng tốn gấp đôi bộ nhớ cho con trỏ và code
// thao tác phức tạp hơn.

void insertHead(DLinkedList& list, int val) {
    DNode* p = new DNode{val, nullptr, nullptr};
    if (!list.pHead) list.pHead = list.pTail = p;
    else { p->pNext = list.pHead; list.pHead->pPrev = p; list.pHead = p; }
}

```

6. Complexity Analysis & Edge Cases

- Chèn/Xóa ở Đầu: $\Theta(1)$ thời gian, $\mathcal{O}(1)$ bộ nhớ.
- Truy xuất theo chỉ số / Tìm kiếm: $\Theta(n)$ thời gian.
- Edge Cases bắt buộc kiểm tra: Danh sách rỗng (`pHead == nullptr`), Danh sách chỉ có 1 nút (`pHead == pTail`).

7. Dry Run Table (Chạy tay Thao tác Add Head & Delete Head)

Thao tác	Trạng thái pHead trước	Trạng thái pTail trước	Biến đổi con trỏ	Trạng thái pHead sau
Add Head(10) vào List rỗng	nullptr	nullptr	<code>pHead = pTail = pNew</code>	Node(10)
Add Head(5) vào List(10)	Node(10)	Node(10)	<code>pNew->pNext = pHead; pHead = pNew</code>	Node(5) $\Rightarrow$ Node(10)
Delete Head trên List(5, 10)	Node(5)	Node(10)	<code>temp=pHead; pHead=pHead->pNext; delete temp</code>	Node(10)

8. Common Errors & IT003 Exam Style

IT003 EXAM STYLE

Bẫy Đề Thi: Khi xóa nút duy nhất trong danh sách, nếu chỉ cập nhật `pHead = nullptr` mà không cập nhật `pTail = nullptr`, con trỏ `pTail` sẽ trở thành **Con trỏ rác (Dangling Pointer)** dẫn đến sập chương trình ở các thao tác sau!

9. Quick Recall & Exercise Ladder

- **Level 0:** Tại sao chèn vào cuối DSLK đơn có con trỏ `pTail` tốn $\mathcal{O}(1)$ nhưng xóa phần tử cuối vẫn tốn $\Theta(n)$?
Đáp án: Vì để cập nhật `pTail` mới, ta phải duyệt từ `pHead` đến nút áp cuối (nút trước `pTail`), tốn $\Theta(n)$.
- **Level 1:** Viết hàm đếm số phần tử trong DSLK đơn.
Gợi ý: Dùng con trỏ chạy `Node* p = list.pHead`, vừa duyệt vừa tăng biến đếm `count++` cho đến khi `p == nullptr`.



Tóm kết Chương 4

DSLK cho phép chèn/xóa ở đầu trong $\mathcal{O}(1)$ mà không cần dời bộ nhớ. Luôn chú ý quản lý cả `pHead` và `pTail` cùng các trường hợp rỗng/1 nút!

NGĂN XẾP VÀ HÀNG ĐỢI (STACK & QUEUE)

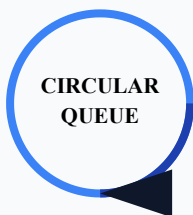
1. Intuition (Trực giác & Động lực)

Stack (Ngăn xếp) giống như một chồng đĩa: đĩa nào đặt vào sau cùng sẽ được lấy ra đầu tiên (LIFO — Last In, First Out). Queue (Hàng đợi) giống như hàng người xếp hàng mua vé: ai đến trước sẽ được phục vụ trước (FIFO — First In, First Out).

2. Formal Model (Định nghĩa Cấu trúc)

- Stack ADT:** Các thao tác chính gồm `push(x)` (thêm vào đỉnh), `pop()` (lấy khỏi đỉnh), `top()` (xem đỉnh), `isEmpty()`.
- Queue ADT:** Các thao tác chính gồm `enqueue(x)` (thêm vào đuôi), `dequeue()` (lấy khỏi đầu), `front()` (xem đầu), `isEmpty()`.

3. Visual Diagram (Sơ đồ Tràn Ảo & Hàng Đợi Vòng)



Công thức Hàng Đợi Vòng (Modulo):

$rear = (rear + 1) \% MAX;$

$front = (front + 1) \% MAX;$

4. Applications & Algorithms

4.1 Hiện tượng Tràn Ảo - False Overflow

Khi dùng mảng tĩnh cho Queue, sau một số thao tác enqueue và dequeue, chỉ số `rear` tiến tới `MAX - 1`. Khi này mảng báo đầy (`rear == MAX - 1`) dù các ô đầu mảng đã bị bỏ trống. Giải pháp là dùng **Circular Queue** với phép toán chia lấy dư `% MAX`.

4.2 Balanced Parentheses Checking

Kiểm tra dấu ngoặc hợp lệ bằng Stack. Gặp mở ngoặc thì push, gặp đóng ngoặc thì pop và kiểm tra khớp loại.

Char	Stack	Thao tác
'('	['(']	Push

)'	[]	Pop và Khớp
----	----	-------------

4.3 Infix to Postfix Conversion

Chuyển biểu thức trung tố sang hậu tố dùng Stack lưu toán tử.

Ký tự đọc	Stack	Output
'A'	[]	"A"
'+'	['+']	"A"
'B'	['+']	"A B"
End of String	[] (Pop '+')	"A B +"

5. Multiple Implementations & STL Code

5.1 Array Implementation

```
// Array Stack
const int MAX = 100;
struct ArrayStack {
    int top_idx = -1;
    int a[MAX];
    void push(int x) { if (top_idx < MAX-1) a[++top_idx] = x; }
    int pop() {
        if (top_idx < 0) return -1; // Underflow check
        return a[top_idx--];
    }
};

// Circular Queue Implementation
struct CircularQueue {
    int front = 0, rear = -1, count = 0;
    int a[MAX];
    void enqueue(int x) {
        if (count == MAX) return; // Overflow check
        rear = (rear + 1) % MAX;
        a[rear] = x; count++;
    }
    int dequeue() {
        if (count == 0) return -1; // Underflow check
        int val = a[front];
        front = (front + 1) % MAX;
        count--;
        return val;
    }
};
```

5.2 Linked List Implementation

```
struct Node { int data; Node* next; };
struct LinkedStack {
    Node* head = nullptr;
    void push(int x) { head = new Node{x, head}; }
    void pop() {
        if (head == nullptr) return;
        Node* t = head; head = head->next; delete t;
    }
};
```

5.3 C++ STL Standard

```
#include <stack>
#include <queue>
void stlStackQueue() {
    std::stack<int> s; s.push(10); s.pop();
    std::queue<int> q; q.push(20); q.pop();
}
```

6. Complexity Analysis & Invariants

Tất cả các thao tác `push`, `pop`, `top`, `enqueue`, `dequeue`, `front` trên Stack và Queue chuẩn đều đạt độ phức tạp $\Theta(1)$ thời gian và $\mathcal{O}(1)$ bộ nhớ phụ.

7. Dry Run Table (Trace Chuỗi Thao Tác Circular Queue)

Thực hiện lần lượt: Enqueue(5), Enqueue(8), Dequeue(), Enqueue(12) trên Queue $MAX = 3$:

Thao tác	front	rear	count	Mảng a[0..2]	Kết quả trả về
Ban đầu	0	-1	0	[-, -, -]	-
Enqueue(5)	0	0	1	[5, -, -]	True
Enqueue(8)	0	1	2	[5, 8, -]	True
Dequeue()	1	1	1	[-, 8, -]	5
Enqueue(12)	1	2	2	[-, 8, 12]	True

8. Common Errors & IT003 Exam Style

IT003 EXAM STYLE

Bẫy Đề Thi: Ứng dụng kiểm tra biểu thức ngoặc hợp lệ (Parentheses Matching) hoặc chuyển đổi Hậu tố (Postfix). Luôn nhớ: DFS dùng **Stack**, BFS dùng **Queue**!

9. Quick Recall & Exercise Ladder

- **Level 0:** Phân biệt nguyên lý hoạt động của Stack và Queue.
Đáp án: Stack là LIFO (Vào sau Ra trước), Queue là FIFO (Vào trước Ra trước).
- **Level 1:** Cho chuỗi thao tác Stack: `Push(3), Push(7), Pop(), Push(5), Pop()`. Giá trị còn lại trong Stack là gì?
Gợi ý: $\text{Push } 3 \Rightarrow \text{Push } 7 \Rightarrow \text{Pop } 7 \Rightarrow \text{Push } 5 \Rightarrow \text{Pop } 5 \Rightarrow \text{Còn lại giá trị } 3$.

Tóm kết Chương 5

Stack và Queue là 2 cấu trúc dữ liệu tuyến tính quan trọng nhất làm tiền đề cho các thuật toán duyệt cây, duyệt đồ thị BFS/DFS.

PART VI

CÂY VÀ CÂY NHỊ PHÂN (TREES & BINARY TREES)

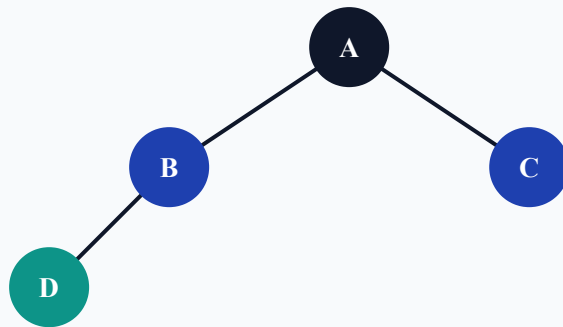
1. Intuition (Trực giác & Động lực)

Khác với mảng hay danh sách liên kết là cấu trúc dữ liệu tuyến tính (Linear), Cây (Tree) là cấu trúc dữ liệu phi tuyến (Non-linear) phân cấp. Cây mô phỏng hoàn hảo các hệ thống phân cấp thực tế như cây thư mục máy tính, sơ đồ tổ chức công ty hay cây quyết định.

2. Formal Model & Terminology (Định nghĩa & Thuật ngữ)

- **Root (Gốc):** Nút duy nhất không có nút cha.
- **Leaf (Nút lá):** Nút không có con nào (bậc của nút bằng 0).
- **Degree (Bậc):** Số nút con của một nút.
- **IT003 CONVENTION** **Level (Mức):** Nút gốc Root nằm ở **Level 0**. Nút con của Root nằm ở Level 1. Chiều cao của cây $H = \max_level + 1$.

3. Visual Diagram (Sơ đồ Duyệt Cây Nhị Phân)



4. Algorithm & Mechanics (3 Phép Duyệt Cây)

- NLR (Pre-order / Tiền tự): Thăm Nút gốc $N \Rightarrow$ Duyệt Cây con trái $L \Rightarrow$ Duyệt Cây con phải R .
- LNR (In-order / Trung tự): Duyệt Cây con trái $L \Rightarrow$ Thăm Nút gốc $N \Rightarrow$ Duyệt Cây con phải R .
- LRN (Post-order / Hậu tự): Duyệt Cây con trái $L \Rightarrow$ Duyệt Cây con phải $R \Rightarrow$ Thăm Nút gốc N .

5. C++ Educational & Traversals Code

```
struct TreeNode {
    int data;
    TreeNode* pLeft;
    TreeNode* pRight;
};

void preOrder(TreeNode* root) {
    if (root != nullptr) {
        std::cout << root->data << " ";
        preOrder(root->pLeft);
        preOrder(root->pRight);
    }
}

void inOrder(TreeNode* root) {
    if (root != nullptr) {
        inOrder(root->pLeft);
        std::cout << root->data << " ";
        inOrder(root->pRight);
    }
}

void postOrder(TreeNode* root) {
    if (root != nullptr) {
        postOrder(root->pLeft);
        postOrder(root->pRight);
        std::cout << root->data << " ";
    }
}
```

```
}  
}
```

6. Complexity Analysis & Invariants

Tất cả các phép duyệt cây nhị phân đều ghé thăm mỗi nút đúng 1 lần, độ phức tạp là $\Theta(n)$ thời gian và $\mathcal{O}(h)$ bộ nhớ phụ (với h là chiều cao cây do gọi đệ quy trên Call Stack).

7. Dry Run Table (Trace Phép Duyệt NLR, LNR, LRN)

Cho cây nhị phân: Gốc A, con trái B, con phải C. B có con trái D:

Phép duyệt	Thứ tự nút được ghé thăm
NLR (Tiền tự)	$A \Rightarrow B \Rightarrow D \Rightarrow C$
LNR (Trung tự)	$D \Rightarrow B \Rightarrow A \Rightarrow C$
LRN (Hậu tự)	$D \Rightarrow B \Rightarrow C \Rightarrow A$

8. Common Errors & IT003 Exam Style

IT003 EXAM STYLE

Bẫy Đề Thi: Xác định chiều cao của cây H . Đọc kỹ quy ước trong đề thi! Nếu đề thi tính Root ở Level 0 thì $H = \text{max_level} + 1$. Nếu tính Root ở Level 1 thì $H = \text{max_level}$.

9. Quick Recall & Exercise Ladder

- Level 0:** Phép duyệt nào trên Cây Nhị Phân Tìm Kiếm (BST) sẽ xuất ra các giá trị khóa theo thứ tự tăng dần?
Đáp án: Phép duyệt Trung tự **LNR (In-order)**.
- Level 1:** Tính số nút tối đa của một Cây nhị phân có chiều cao h (Root ở Level 0, height $h = \text{max_level} + 1$).
Gợi ý: Số nút tối đa là $2^h - 1$.

Tóm kết Chương 6

Nắm vững khái niệm mức (Level), chiều cao và 3 phép duyệt NLR, LNR, LRN là chìa khóa để xử lý toàn bộ các câu hỏi về Cây trong đề thi IT003.

CÂY NHỊ PHÂN TÌM KIẾM (BINARY SEARCH TREE - BST)

1. Intuition (Trực giác & Động lực)

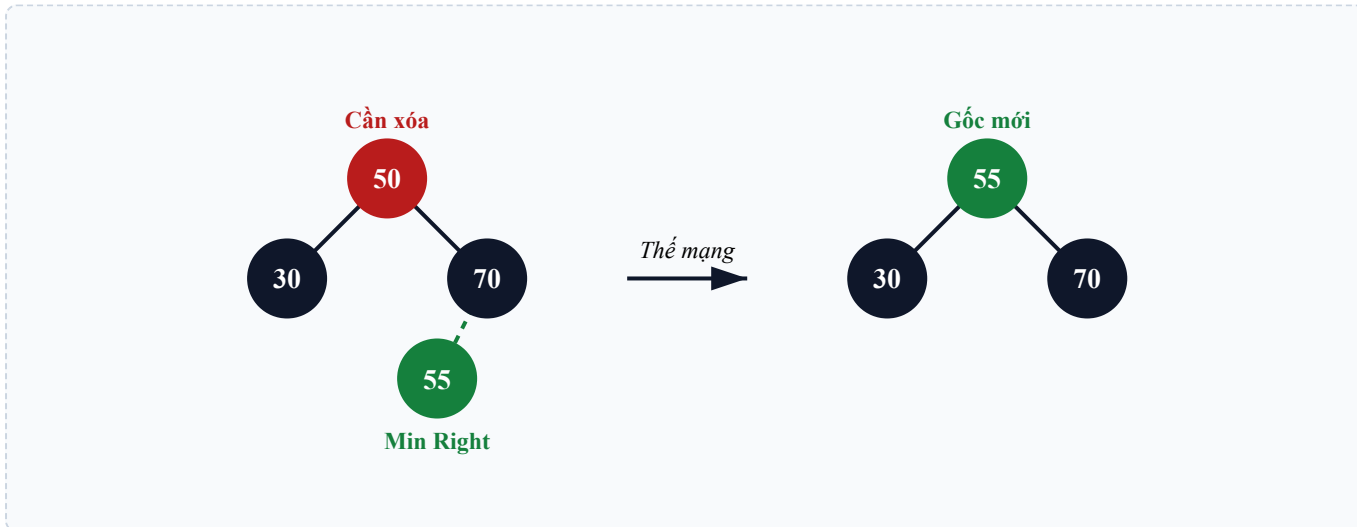
Mảng sắp xếp cho phép Tìm kiếm Nhị phân $\mathcal{O}(\log n)$ nhưng chèn/xóa tốn $\mathcal{O}(n)$. Danh sách liên kết cho phép chèn/xóa $\mathcal{O}(1)$ nhưng tìm kiếm tốn $\mathcal{O}(n)$. Với **Binary Search Tree (BST)**, Tìm kiếm, Chèn và Xóa đều là $\mathcal{O}(h)$ theo chiều cao cây: trung bình $\mathcal{O}(\log n)$ khi cây cân đối, nhưng Worst Case $\mathcal{O}(n)$ khi cây lệch.

2. Formal Model & Invariants (Định nghĩa & Bất biến)

BST là cây nhị phân thỏa mãn tính chất bất biến tại mọi nút X :

- Mọi khóa ở cây con trái của X đều **nhỏ hơn** khóa của X ($\text{key}(\text{Left}) < \text{key}(X)$).
- Mọi khóa ở cây con phải của X đều **lớn hơn** khóa của X ($\text{key}(\text{Right}) > \text{key}(X)$).

3. Visual Diagram (Sơ đồ Xóa Nút BST 2 Con)



4. Algorithm & Mechanics (Các thao tác cơ bản)

- Tìm kiếm (Search):** Duyệt từ gốc, nhỏ sang trái, lớn sang phải. $\mathcal{O}(h)$; cây cân đối là $\mathcal{O}(\log n)$, cây lệch có thể là $\mathcal{O}(n)$.
- Thêm (Insert):** Tương tự tìm kiếm để tìm vị trí nút lá thích hợp rồi chèn.
- Min/Max:** Min nằm ở nút cực trái, Max nằm ở nút cực phải.
- Duyệt cây (Traversals):**
 - Pre-order (NLR): Root $\rightarrow$ Left $\rightarrow$ Right
 - In-order (LNR): Left $\rightarrow$ Root $\rightarrow$ Right (In ra mảng tăng dần)
 - Post-order (LRN): Left $\rightarrow$ Right $\rightarrow$ Root

- Xóa (Delete) - 3 Trường hợp:
 - Nút lá: Xóa trực tiếp nút.
 - Nút có 1 con: Nối cha của nút cần xóa trực tiếp tới nút con của nó.
 - Nút có 2 con: **IT003 CONVENTION** Thay bằng Nút cực phải của cây con trái (Max Left) hoặc Nút cực trái của cây con phải (Min Right).

5. C++ Educational Implementation

```
// Search (Recursive)
TreeNode* search(TreeNode* root, int key) {
    if (!root || root->data == key) return root;
    if (key < root->data) return search(root->pLeft, key);
    return search(root->pRight, key);
}

// Search (Iterative)
TreeNode* searchIterative(TreeNode* root, int key) {
    while (root != nullptr && root->data != key) {
        if (key < root->data) root = root->pLeft;
        else root = root->pRight;
    }
    return root;
}

// LNR Traversal
void inOrder(TreeNode* root) {
    if (root) {
        inOrder(root->pLeft);
        std::cout << root->data << " ";
        inOrder(root->pRight);
    }
}

TreeNode* findMin(TreeNode* root) {
    while (root && root->pLeft != nullptr) root = root->pLeft;
    return root;
}

TreeNode* findMax(TreeNode* root) {
    while (root && root->pRight != nullptr) root = root->pRight;
    return root;
}

TreeNode* deleteNode(TreeNode* root, int key) {
    if (root == nullptr) return root;

    if (key < root->data) root->pLeft = deleteNode(root->pLeft, key);
    else if (key > root->data) root->pRight = deleteNode(root->pRight, key);
    else {
        if (root->pLeft == nullptr) {
            TreeNode* temp = root->pRight;
            delete root; return temp;
        } else if (root->pRight == nullptr) {
```

```
        TreeNode* temp = root->pLeft;
        delete root; return temp;
    }
    TreeNode* temp = findMin(root->pRight); // Min Right
    root->data = temp->data;
    root->pRight = deleteNode(root->pRight, temp->data);
}
return root;
}
```

6. Complexity Analysis

- Các thao tác Tìm kiếm, Chèn, Xóa trên BST có độ phức tạp phụ thuộc trực tiếp vào chiều cao của cây: $\mathcal{O}(h)$.
- Cây cân bằng tương đối ($h = \Theta(\log n)$): Các thao tác đạt hiệu năng lý tưởng $\mathcal{O}(\log n)$.
 - Cây suy biến tệ nhất (Skewed Tree, $h = n$): Cây bị suy biến thành Danh sách liên kết, các thao tác tốn $\mathcal{O}(n)$.

7. Dry Run Table (Trace Thêm và Xóa Nút)

Trace Thêm lần lượt 50, 30, 70, 20:

Khóa chèn	Hành động	Kết quả Cây
50	Cây rỗng, tạo Root 50	Root: 50
30	$30 < 50$, chèn trái 50	$50 \Rightarrow \text{L:}30$
70	$70 > 50$, chèn phải 50	$50 \Rightarrow \text{L:}30, \text{R:}70$
20	$20 < 50$ (L), $20 < 30$ (L), chèn trái 30	$50 \Rightarrow \text{L:}30(\text{L:}20), \text{R:}70$

Trace Xóa Nút 50 (có 2 con, dùng Min Right):

Bước	Hành động	Trạng thái Nút 50	Nút thế mạng Min Right
1	Tìm nút 50	Xác định có 2 con (Left 30, Right 70)	-
2	Tìm Min Right trên cây con 70	-	Tìm thấy Nút 55
3	Gán <code>root->data = 55</code>	Dữ liệu đổi thành 55	-
4	Đệ quy xóa 55 trên cây con phải	Nút 50 cũ giữ giá trị 55	Xóa nút 55 ban đầu

8. Common Errors & IT003 Exam Style

IT003 EXAM STYLE

Bẫy Đề Thi: Luôn đọc kỹ yêu cầu bài thi: "Thay bằng phần tử lớn nhất bên cây con trái" (Max Left) hay "nhỏ nhất bên cây con phải" (Min Right). Chọn sai phần tử thế mạng sẽ bị trừ điểm toàn bộ cây kết quả!

9. Quick Recall & Exercise Ladder

- Level 0:** BST bị suy biến thành danh sách tuyến tính khi nào?
Đáp án: Khi chèn các phần tử theo thứ tự đã sắp xếp tăng dần hoặc giảm dần.
- Level 1:** Vẽ BST kết quả sau khi chèn dãy khóa: 50, 30, 70, 20, 40, 60, 80.
Gợi ý: 50 là Root. 30 bên trái 50, 70 bên phải 50, ... thu được cây cân bằng hoàn hảo.

Tóm kết Chương 7

BST là nền tảng quan trọng nhất của cấu trúc cây tìm kiếm. Hãy luyện tập thành thạo thao tác xóa nút 2 con trên giấy!

PART VIII

CÂY CÂN BẰNG AVL (AVL TREES)

1. Intuition (Trực giác & Động lực)

Để khắc phục nhược điểm bị suy biến thành $\mathcal{O}(n)$ của BST thông thường, Adelson-Velsky và Landis (AVL) đã đề xuất cây BST tự cân bằng: tự động thực hiện phép **Xoay (Rotation)** mỗi khi độ lệch chiều cao giữa 2 cây con vượt quá 1.

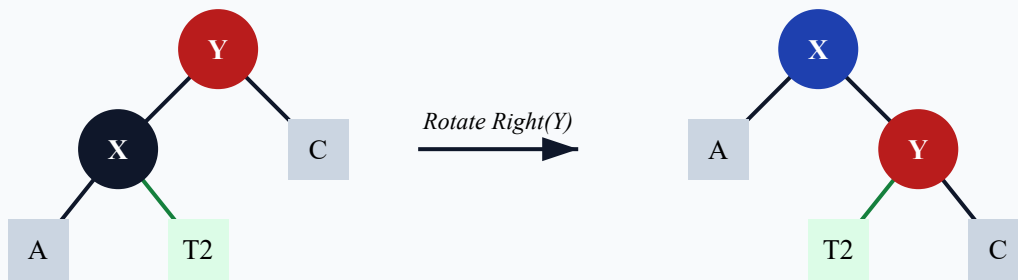
2. Formal Model & Balance Factor

Chỉ số cân bằng của nút X :

$$BF(X) = h_{left} - h_{right}$$

Điều kiện AVL: $BF(X) \in \{-1, 0, +1\}$ với mọi nút X . Nếu $|BF(X)| \geq 2$, cây bị mất cân bằng tại X .

3. Visual Diagram (Cơ chế Xoay Đơn Phải - Right Rotation)



Rotation Cheat Sheet Table

Trường hợp	$BF(A)$ và $BF(Child)$	Phép Xoay Khôi Phục
LL (Left-Left)	$BF(A) = +2, BF(Left) \geq 0$	Xoay Đơn Phải (Rotate Right) tại A
RR (Right-Right)	$BF(A) = -2, BF(Right) \leq 0$	Xoay Đơn Trái (Rotate Left) tại A
LR (Left-Right)	$BF(A) = +2, BF(Left) = -1$	Xoay Kép LR (Rotate Left tại Left, rồi Rotate Right tại A)
RL (Right-Left)	$BF(A) = -2, BF(Right) = +1$	Xoay Kép RL (Rotate Right tại Right, rồi Rotate Left tại A)

4. Algorithm & Mechanics (Xoay Đơn Phải Rotate Right)

Cho nút Y bị mất cân bằng nghiêng trái ($BF = +2$), nút con trái là X . Phép xoay phải đưa X lên làm cha Y , và chuyển cây con phải của X (T_2) sang làm cây con trái của Y .

Tại sao phép xoay bảo toàn tính chất BST?

Theo tính chất BST ban đầu: $A < X < T_2 < Y < C$. Khi xoay phải, X trở thành gốc mới, Y trở thành con phải của X . Nút T_2 (lớn hơn X nhưng nhỏ hơn Y) được chuyển một cách hoàn toàn tự nhiên từ con phải của X sang làm con trái của Y , vẫn thỏa mãn chặt chẽ bất biến BST.

5. C++ Educational Implementation

```
#include <algorithm>

struct AVLNode {
    int key;
```

```

    int height;
    AVLNode* left;
    AVLNode* right;
};

int getHeight(AVLNode* n) { return n == nullptr ? 0 : n->height; }
int getBalance(AVLNode* n) { return n == nullptr ? 0 : getHeight(n->left) - getHeight(n->right); }

AVLNode* rotateRight(AVLNode* y) {
    AVLNode* x = y->left;
    AVLNode* T2 = x->right;
    x->right = y;
    y->left = T2;
    y->height = std::max(getHeight(y->left), getHeight(y->right)) + 1;
    x->height = std::max(getHeight(x->left), getHeight(x->right)) + 1;
    return x;
}

AVLNode* rotateLeft(AVLNode* x) {
    AVLNode* y = x->right;
    AVLNode* T2 = y->left;
    y->left = x;
    x->right = T2;
    x->height = std::max(getHeight(x->left), getHeight(x->right)) + 1;
    y->height = std::max(getHeight(y->left), getHeight(y->right)) + 1;
    return y;
}

AVLNode* insertAVL(AVLNode* node, int key) {
    if (node == nullptr) return new AVLNode{key, 1, nullptr, nullptr};
    if (key < node->key) node->left = insertAVL(node->left, key);
    else if (key > node->key) node->right = insertAVL(node->right, key);
    else return node;

    node->height = 1 + std::max(getHeight(node->left), getHeight(node->right));
    int balance = getBalance(node);

    // LL Case
    if (balance > 1 && key < node->left->key) return rotateRight(node);
    // RR Case
    if (balance < -1 && key > node->right->key) return rotateLeft(node);
    // LR Case
    if (balance > 1 && key > node->left->key) {
        node->left = rotateLeft(node->left);
        return rotateRight(node);
    }
    // RL Case
    if (balance < -1 && key < node->right->key) {
        node->right = rotateRight(node->right);
        return rotateLeft(node);
    }
    return node;
}

```

6. Complexity Analysis

Cây AVL luôn đảm bảo chiều cao tiệm cận $h = \Theta(\log n)$. Do đó các thao tác Tìm kiếm, Chèn, Xóa luôn luôn đạt $\Theta(\log n)$ trong mọi trường hợp.

7. Dry Run Table (Trace 4 Trường Hợp Mất Cân Bằng Cơ Bản)

Dưới đây là 4 kịch bản chèn 3 khóa gây mất cân bằng điển hình:

Trường hợp	Các khóa chèn	Trạng thái cây (Nghiêng)	Khắc phục (Phép xoay)	Kết quả (Root, L, R)
LL	30, 20, 10	$30 \Rightarrow L:20 \Rightarrow L:10$	Xoay Phải (30)	Root:20, L:10, R:30
RR	10, 20, 30	$10 \Rightarrow R:20 \Rightarrow R:30$	Xoay Trái (10)	Root:20, L:10, R:30
LR	30, 10, 20	$30 \Rightarrow L:10 \Rightarrow R:20$	Xoay Trái (10), Xoay Phải (30)	Root:20, L:10, R:30
RL	10, 30, 20	$10 \Rightarrow R:30 \Rightarrow L:20$	Xoay Phải (30), Xoay Trái (10)	Root:20, L:10, R:30

8. Common Errors & IT003 Exam Style

IT003 EXAM STYLE

Bẫy Đề Thi: Nhầm lẫn giữa xoay đơn và xoay kép. Dấu hiệu xoay kép LR: $BF(A) = +2$ nhưng con trái lại có $BF = -1$. Phải xoay trái con trái trước, sau đó mới xoay phải A!

9. Quick Recall & Exercise Ladder

- Level 0:** Chiều cao cây con trái là 3, cây con phải là 1. Chỉ số cân bằng BF của gốc là bao nhiêu? Cây có đạt chuẩn AVL không?
Đáp án: $BF = 3 - 1 = +2$. Cây không đạt chuẩn AVL (cần xoay).
- Level 1:** Chèn lần lượt các số 1, 2, 3 vào cây AVL rỗng. Xác định loại xoay.
Gợi ý: Mất cân bằng RR tại nút 1 $\Rightarrow$ Xoay Trái nút 1.

Tóm kết Chương 8

Cây AVL duy trì sự cân bằng nghiêm ngặt nhờ 4 phép xoay chuẩn (LL, RR, LR, RL), đảm bảo thời gian truy xuất lý tưởng $\mathcal{O}(\log n)$.

HEAPS VÀ HÀNG ĐỢI ƯU TIÊN (HEAP & PRIORITY QUEUE)

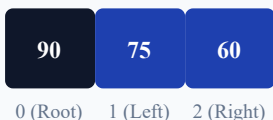
1. Intuition (Trực giác & Động lực)

Hàng đợi ưu tiên (Priority Queue) là hàng đợi mà phần tử có độ ưu tiên cao nhất luôn được phục vụ trước. Cấu trúc dữ liệu Heap biểu diễn hoàn hảo hàng đợi ưu tiên nhờ khả năng tìm phần tử cực đại/cực tiểu trong $\mathcal{O}(1)$ và cập nhật lại cây trong $\mathcal{O}(\log n)$.

2. Formal Model & Invariants

Max-Heap là Cây nhị phân hoàn chỉnh (Complete Binary Tree) thỏa mãn bất biến: Khóa của nút cha luôn $\geq$ khóa của các nút con.

3. Visual Diagram (Biểu diễn Heap bằng Mảng)



4. Algorithm & Mechanics (Công thức Chỉ số Mảng)

IT003 CONVENTION : Với mảng bắt đầu từ index 0 :

- $\text{parent}(i) = \lfloor (i - 1) / 2 \rfloor$
- $\text{left}(i) = 2i + 1$
- $\text{right}(i) = 2i + 2$

5. C++ Educational & STL Implementation

```
#include <algorithm>
#include <queue>

void heapify(int a[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1, right = 2 * i + 2;
    if (left < n && a[left] > a[largest]) largest = left;
    if (right < n && a[right] > a[largest]) largest = right;
    if (largest != i) {
        swap(a[i], a[largest]);
        heapify(a, n, largest);
    }
}
```

```

    if (right < n && a[right] > a[largest]) largest = right;
    if (largest != i) {
        std::swap(a[i], a[largest]);
        heapify(a, n, largest);
    }
}

// C++ STL Priority Queue
void stlPriorityQueue() {
    std::priority_queue<int> maxHeap; // Max Heap mặc định
    maxHeap.push(50); maxHeap.push(80);
    int topVal = maxHeap.top(); // 80
}

```

6. Complexity Analysis

Tìm Max: $\Theta(1)$. Chèn / Xóa Max (Heapify): $\Theta(\log n)$. Thuật toán `buildHeap` tạo Heap từ mảng ngẫu nhiên đạt độ phức tạp tuyến tính $\Theta(n)$.

Giải thích trực giác vì sao buildHeap là $\Theta(n)$: Hầu hết các nút nằm ở đáy cây (mức lá). Lá không cần heapify (chiều cao $h = 0$). Chỉ có một vài nút ở đỉnh cây mới cần rớt xuống sâu. Tổng số thao tác rớt của tất cả các nút tạo thành một chuỗi giảm dần hội tụ về $\Theta(n)$. Heap Sort (Build Heap + Extract Max liên tục) đạt $\Theta(n \log n)$.

7. Dry Run Table (Trace Heap Sort)

Thuật toán Heap Sort thực hiện các bước: BuildHeap $\rightarrow$ Extract Max $\rightarrow$ Swap với phần tử cuối $\rightarrow$ Shrink kích thước $\rightarrow$ Heapify root.

Sắp xếp $A = [10, 50, 40]$ (đã là Max-Heap sau buildHeap với kết quả $A = [50, 10, 40]$):

Bước	Mảng hiện tại	Hành động	Kích thước Heap
1	[50, 10, 40]	Đổi chỗ A[0] (50) và A[2] (40)	Giảm xuống $n=2$
2	[40, 10, 50]	Heapify(A, 2, 0): $40 > 10$, giữ nguyên	$n=2$
3	[40, 10, 50]	Đổi chỗ A[0] (40) và A[1] (10)	Giảm xuống $n=1$
4	[10, 40, 50]	Hoàn thành, mảng đã sắp xếp tăng dần	$n=1$

8. Common Errors & IT003 Exam Style

IT003 EXAM STYLE

Bẫy Đề Thi: Cây nhị phân hoàn chỉnh phải được điền đầy từ trái sang phải ở mức cuối cùng. Đừng nhầm lẫn giữa Max-Heap và Cây BST!

9. Quick Recall & Exercise Ladder

- **Level 0:** Trong Max-Heap dạng mảng, nút tại vị trí index 3 có nút con trái ở index bao nhiêu?
Đáp án: Index $2(3) + 1 = 7$.
- **Level 1:** Cho mảng $A = [4, 10, 3, 5, 1]$. Chạy tay `buildHeap` để biến A thành Max-Heap.
Gợi ý: Gọi Heapify từ vị trí $i = \lfloor n/2 \rfloor - 1 = 1 \Rightarrow A = [10, 5, 3, 4, 1]$.

Tóm kết Chương 9

Heap cung cấp cấu trúc ưu tiên hiệu quả nhất với chi phí $\mathcal{O}(\log n)$ cho thao tác cập nhật và $\mathcal{O}(1)$ cho thao tác xem phần tử ưu tiên nhất.

PART X

CÂY B-TREE (B-TREES - MULTI-WAY SEARCH TREES)

1. Intuition (Trực giác & Động lực)

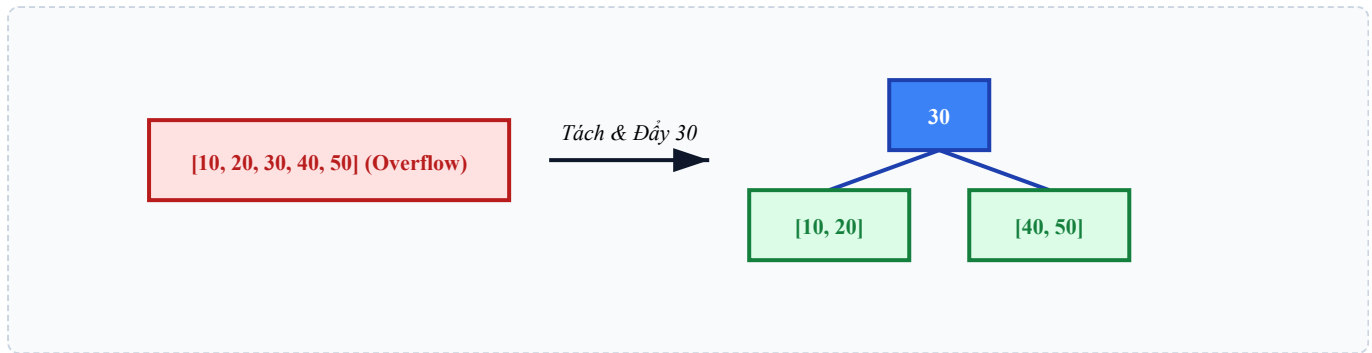
Khi dữ liệu quá lớn không thể chứa hết trong RAM mà phải lưu trên đĩa cứng (Hard Drive / SSD), mỗi lần đọc đĩa (Disk I/O) tốn thời gian rất lớn. **B-Tree (Cây tìm kiếm đa đường)** làm giảm chiều cao của cây bằng cách cho phép mỗi nút chứa nhiều khóa và nhiều con, tối ưu hóa tối đa số lần đọc đĩa.

2. Formal Model & Rules (Quy tắc B-Tree Bậc $m = 5$)

IT003 CONVENTION : B-Tree bậc 5 ($m = 5$) xuất hiện phổ biến trong các bộ đề IT003 đã phân tích với các quy chuẩn:

- Số khóa tối đa trong 1 nút: $m - 1 = 4$ khóa.
- Số con tối đa của 1 nút: $m = 5$ con.
- Số khóa tối thiểu (nút không phải gốc): $\lceil m/2 \rceil - 1 = \lceil 2.5 \rceil - 1 = 2$ khóa.
- Số con tối thiểu (nút không phải gốc): $\lceil m/2 \rceil = 3$ con.
- Tất cả các nút lá đều nằm trên cùng một mức.

3. Visual Diagram (Sơ đồ Tách Nút B-Tree Bậc 5)



4. Algorithm & Mechanics (Tách Nút và Xóa Nút)

Thêm (Insert) & Tách Nút (Split)

Khi chèn vào nút đã có 4 khóa làm xuất hiện khóa thứ 5 ($[k_1, k_2, k_3, k_4, k_5]$): Khóa trung vị k_3 được **đẩy nổi** (Promote) lên nút cha. Nút bị **Tách** (Split) thành 2 nút con chứa $[k_1, k_2]$ và $[k_4, k_5]$.

Xóa (Delete) trên B-Tree bậc 5 ($m=5$)

- Trường hợp 1: Xóa ở nút lá và $\geq$ min keys (Số khóa ≥ 2): Xóa trực tiếp, không vi phạm cấu trúc.
- Trường hợp 2: Xóa gây Underflow (Số khóa tụt xuống 1):
 - Mượn (Borrow / Redistribution): Nếu anh em ruột kề cạnh có > 2 khóa, mượn 1 khóa thông qua nút cha. Khóa của anh em lên cha, khóa của cha rớt xuống nút hiện tại.
 - Gộp (Merge): Nếu anh em kề cạnh chỉ có đúng 2 khóa, tiến hành gộp nút hiện tại, nút anh em và 1 khóa từ cha lại thành 1 nút (tổng cộng $1 + 2 + 1 = 4$ khóa).
- Xóa ở nút trong (Internal Node): Thay thế bằng phần tử tiền nhiệm (Max Left) hoặc kế nhiệm (Min Right) ở nút lá, sau đó tiến hành xóa ở nút lá đó.
- Trường hợp Đặc biệt (Root): Nếu gốc sau khi Merge bị mất hết khóa (chỉ còn 1 con), nút con đó sẽ trở thành Root mới. Chiều cao cây giảm đi 1.

5. C++ Pseudocode Representative Structure

```
const int M = 5; // B-Tree Order 5
struct BTreeNode {
    int keys[M - 1]; // Tối đa 4 khóa
    BTreeNode* child[M]; // Tối đa 5 con
    int numKeys;
    bool isLeaf;
};
```

6. Complexity Analysis

Thời gian Tìm kiếm, Chèn, Xóa trên B-Tree luôn đạt $\Theta(\log_m n)$, giúp giảm số lần đọc đĩa xuống mức tối thiểu.

7. Dry Run Table (Trace B-Tree bậc 5)

Trace 1: Thêm Khóa (Split)

Chèn 10, 20, 30, 40, 50 vào B-Tree bậc 5 rỗng:

Khóa chèn	Trạng thái Nút lá	Hiện tượng	Hành động Tách & Đẩy nổi
10, 20, 30, 40	[10, 20, 30, 40]	Đủ 4 khóa (Bình thường)	Chưa tách
50	[10, 20, 30, 40, 50]	Tràn (5 khóa)	Đẩy nổi 30 lên Root. Tách thành 2 con: [10, 20] và [40, 50]

Trace 2: Xóa Khóa (Borrow / Redistribution)

Giả sử Root: [30]. Left: [10, 20]. Right: [40, 50, 60]. Cần xóa 10.

Nút Xóa	Trạng thái sau xóa	Khắc phục (Mượn từ Right qua Root)
Xóa 10	Left còn [20] (Underflow, < 2 khóa)	Right có 3 khóa (dư dả). 40 đẩy lên Root, 30 rớt xuống Left.
Kết quả	-	Root: [40]. Left: [20, 30]. Right: [50, 60].

Trace 3: Xóa Khóa (Merge Nodes)

Giả sử Root: [30]. Left: [10, 20]. Right: [40, 50]. Cần xóa 10.

Nút Xóa	Trạng thái sau xóa	Khắc phục (Gộp với Right & Root)
Xóa 10	Left còn [20] (Underflow). Right chỉ có [40, 50] (không cho mượn được)	Gộp Left [20], Root [30], Right [40, 50] thành 1 nút.
Kết quả	-	Root mới: [20, 30, 40, 50]. Chiều cao cây giảm 1.

8. Common Errors & IT003 Exam Style

IT003 EXAM STYLE

Bẫy Đề Thi: Khi tách nút có 5 khóa trong B-Tree bậc 5, khóa ở giữa (vị trí thứ 3) được đưa lên cha. Đừng nhầm lẫn vị trí khóa được đẩy nổi!

9. Quick Recall & Exercise Ladder

- Level 0:** B-Tree bậc 5 chứa tối đa bao nhiêu khóa trong 1 nút? Nút không phải gốc chứa tối thiểu bao nhiêu khóa?

Đáp án: Tối đa $m - 1 = 4$ khóa. Tối thiểu $\lceil 5/2 \rceil - 1 = 2$ khóa.

- Level 1:** Vẽ B-Tree bậc 5 sau khi chèn các số 5, 15, 25, 35, 45, 55.

Gợi ý: Sau khi chèn 45, nút tràn $[5, 15, 25, 35, 45] \Rightarrow$ Đẩy 25 lên gốc. Sau đó chèn 55 vào nút phải $[35, 45, 55]$.

Tóm kết Chương 10

B-Tree bậc 5 quy định chặt chẽ số khóa từ 2 đến 4 khóa per node. Quy tắc Split đẩy nổi khóa thứ 3 là tâm điểm các câu hỏi B-Tree trong đề thi IT003.

PART XI

BẢNG BĂM (HASH TABLE)

1. Intuition (Trực giác & Động lực)

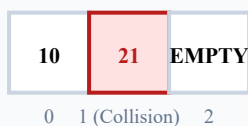
Bạn có muốn tìm kiếm dữ liệu trong thời gian tức thì $\mathcal{O}(1)$ không? Bảng băm (Hash Table) biến khóa k thành chỉ số mảng thông qua **Hàm Băm (Hash Function)**, cho phép truy xuất dữ liệu gần như ngay lập tức.

2. Formal Model & Collision Resolution

Hàm băm modulo: $h(k) = k \bmod M$ (M là kích thước bảng băm). Đụng độ (Collision) xảy ra khi 2 khóa khác nhau có cùng giá trị băm. Các phương pháp giải quyết đụng độ chuẩn IT003:

- Linear Probing (Dò tuyến tính):** $h(k, i) = (h(k) + i) \bmod M$
- Quadratic Probing (Dò bậc hai):** $h(k, i) = (h(k) + i^2) \bmod M$
- Double Hashing (Băm kép):** $h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod M$ (trong đó $h_2(k)$ là hàm băm thứ hai bất buộc tuân theo đề bài cho; ví dụ phổ biến: $h_2(k) = R - (k \bmod R)$).
- Separate Chaining:** Lưu mỗi ô bằng một Danh sách liên kết đơn.

3. Visual Diagram (Sơ đồ Dò Tuyến Tính Hash Probing)



4. Algorithm & Mechanics (Đếm So Sánh C_{succ} và C_{unsucc})

IT003 CONVENTION :

- **Tìm kiếm Thành công (C_{succ}):** Trung bình số phép so sánh để tìm thấy các khóa đang có trong bảng.
- **Tìm kiếm Thất bại (C_{unsucc}):** Trung bình số phép so sánh xuất phát từ mỗi vị trí $i \in [0, M - 1]$, lặp theo **đúng** chuỗi dò thực tế của **phương pháp tương ứng** (Linear / Quadratic / Double Hashing) cho đến khi **gặp ô trống** (**EMPTY**) đầu tiên. Ô bị xóa (**DELETED**) không dừng chuỗi dò!

5. C++ Educational & STL Code

```
#include <unordered_map>
#include <iostream>
#include <string>

void stlHashTable() {
    std::unordered_map<std::string, int> hashMap;
    hashMap["UIT"] = 100;
    if (hashMap.find("UIT") != hashMap.end()) {
        std::cout << "Found: " << hashMap["UIT"] << "\n";
    }
}
```

6. Complexity Analysis

Trung bình: Tìm kiếm / Chèn / Xóa tốn $\Theta(1)$ thời gian. Trường hợp tệ nhất (tất cả các khóa bị đụng độ về 1 ô): $\Theta(n)$.

7. Dry Run Tables (Minh họa các phương pháp Dò)

7.1 Linear Probing ($h(k) = k \bmod 7$)

Chèn 14, 21, 7 vào bảng băm $M = 7$ rỗng:

Khóa k	$h(k) = k \% 7$	Vị trí ô chèn	Số lần so sánh (Probes)	Trạng thái bảng băm [0..6]
14	0	0 (Trống)	1	[14, -, -, -, -, -]
21	0	0 (Trùng) $\Rightarrow$ ô 1 (Trống)	2	[14, 21, -, -, -, -]
7	0	0, 1 (Trùng) $\Rightarrow$ ô 2 (Trống)	3	[14, 21, 7, -, -, -]

7.2 Quadratic Probing ($h(k, i) = (h(k) + i^2) \bmod 7$)

Chèn 14, 21, 28 vào bảng băm $M = 7$ (hàm băm $h(k) = k \bmod 7$):

Khóa k	$h(k)$	Chuỗi dò ($i=0, 1, 2\dots$)	Probes	Trạng thái bảng
14	0	0 (Trống)	1	[14, -, -, -, -, -]
21	0	$0 \Rightarrow (0 + 1^2)\%7 = 1$ (Trống)	2	[14, 21, -, -, -, -]
28	0	$0 \Rightarrow 1 \Rightarrow (0 + 2^2)\%7 = 4$ (Trống)	3	[14, 21, -, -, 28, -, -]

7.3 Double Hashing ($h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod 7$)

Với $h_1(k) = k \bmod 7$ và $h_2(k) = 5 - (k \bmod 5)$. Chèn 14, 21, 7:

Khóa k	$h_1(k), h_2(k)$	Chuỗi dò ($i=0, 1, 2\dots$)	Probes
14	0, 1	0 (Trống)	1
21	0, 4	$0 \Rightarrow (0 + 4)\%7 = 4$ (Trống)	2
7	0, 3	$0 \Rightarrow 3$ (Trống)	2

Đếm số phép so sánh: C_{succ} là trung bình số probes để chèn các khóa thành công (ví dụ trên: $C_{\text{succ}} = (1 + 2 + 2)/3 = 1.67$). C_{unsucc} yêu cầu dò từ từng ô $0 \dots M - 1$ đến khi gặp ô **EMPTY**.

8. Common Errors & IT003 Exam Style

IT003 EXAM STYLE

Bẫy Đề Thi: Tính số phép so sánh tìm kiếm thất bại C_{unsucc} . Phải duyệt từ từng ô index $0, 1, \dots, M - 1$ đếm số bước tới khi gặp ô trống **EMPTY** đầu tiên, sau đó cộng lại chia cho M !

9. Quick Recall & Exercise Ladder

- Level 0:** Trong Linear Probing, khi xóa một phần tử, tại sao ta không được gán ô đó thành **EMPTY** mà phải gán thành **DELETED** ?
Đáp án: Để không ngắt đoạn chuỗi dò của các phần tử đựng độ được chèn sau đó.
- Level 1:** Cho $M = 5$, $h(k) = k \bmod 5$. Chèn 5, 10, 15 dùng Linear Probing. Tính C_{succ} trung bình.
Gợi ý: 5 (1 probe ô 0), 10 (2 probes ô 1), 15 (3 probes ô 2). $C_{\text{succ}} = (1 + 2 + 3)/3 = 2$ phép so sánh.

Tóm kết Chương 11

Bảng băm mang lại hiệu năng $\mathcal{O}(1)$ lý tưởng. Hãy chú ý kỹ quy tắc dừng khi tìm kiếm thất bại và công thức đếm probe!

ĐỒ THỊ VÀ BIỂU DIỄN ĐỒ THỊ (GRAPHS & REPRESENTATIONS)

1. Intuition (Trực giác & Động lực)

Đồ thị (Graph) là mô hình tổng quát nhất biểu diễn các mối quan hệ liên kết: mạng xã hội (user là đỉnh, bạn bè là cạnh), bản đồ giao thông (giao lộ là đỉnh, đường đi là cạnh), hay mạng máy tính.

2. Formal Model & Definitions

Đồ thị $G = (V, E)$ gồm tập đỉnh V ($|V| = n$) và tập cạnh E ($|E| = m$).

- Phân loại:** Vô hướng (Undirected) / Có hướng (Directed), Có trọng số (Weighted) / Không trọng số.
- Bậc (Degree):** Đối với đồ thị vô hướng, $\deg(v)$ là số cạnh nối với đỉnh v . Với đồ thị có hướng, chia làm **Bán bậc vào (Indegree)** $\deg^-(v)$ (số cạnh đi vào) và **Bán bậc ra (Outdegree)** $\deg^+(v)$ (số cạnh đi ra).
- Thành phần liên thông (Connected Components):** Một đồ thị vô hướng là liên thông nếu có đường đi giữa mọi cặp đỉnh. Nếu không, nó phân rã thành nhiều thành phần liên thông. Đối với đồ thị có hướng, ta xét Liên thông mạnh (Strongly Connected) và Liên thông yếu.

3. Visual Diagram (Ma trận Kề vs Danh sách Kề)

Ma trận Kề (Matrix):

$A[u][v] = 1$ nếu có cạnh (u, v)

Bộ nhớ: $O(V^2)$

Danh sách Kề (List):

$\text{adj}[u] = \{v_1, v_2, \dots\}$

Bộ nhớ: $O(V + E)$

4. Algorithm & Mechanics (Chuyển đổi Biểu diễn)

Chuyển đổi từ Ma trận kề $A[V][V]$ sang Danh sách kề: Duyệt qua từng hàng $i \in [0, V - 1]$, nếu $A[i][j] \neq 0$ thì thêm j vào danh sách `adj[i]`.

5. C++ Representations Code

```
#include <vector>
#include <iostream>

// Representation via Adjacency List
void buildGraph(int V) {
```

```
std::vector<std::vector<int>> adj(V);
// Thêm cạnh vô hướng (0, 1)
adj[0].push_back(1);
adj[1].push_back(0);
}
```

6. Complexity Analysis Comparison

Thao tác	Ma trận Kề (Adjacency Matrix)	Danh sách Kề (Adjacency List)
Bộ nhớ (Space)	$\Theta(V^2)$	$\Theta(V + E)$
Kiểm tra cạnh (u, v)	$\Theta(1)$	$\mathcal{O}(\text{degree}(u))$
Tìm tất cả đỉnh kề của u	$\Theta(V)$	$\Theta(\text{degree}(u))$

7. Dry Run Table (Trace Ma trận Kề $\Rightarrow$ Danh sách Kề)

Cho đồ thị 3 đỉnh {0, 1, 2} có Ma trận kề: row0=[0,1,1], row1=[1,0,0], row2=[1,0,0]:

Đỉnh u	Các cột v có $A[u][v] == 1$	Danh sách kề adj[u]
0	1, 2	adj[0] = {1, 2}
1	0	adj[1] = {0}
2	0	adj[2] = {0}

8. Common Errors & IT003 Exam Style

IT003 EXAM STYLE

Bẫy Đề Thi: Phân biệt Đồ thị Có hướng và Vô hướng. Đồ thị Vô hướng có Ma trận kề đối xứng qua đường chéo chính ($A[i][j] = A[j][i]$). Tổng bậc các đỉnh bằng $2E$.

9. Quick Recall & Exercise Ladder

- Level 0:** Đồ thị thưa ($E \ll V^2$) nên biểu diễn bằng ma trận kề hay danh sách kề?
Đáp án: Danh sách kề để tiết kiệm bộ nhớ ($\Theta(V + E)$ thay vì $\Theta(V^2)$).
- Level 1:** Tính tổng số bậc của tất cả các đỉnh trong đồ thị vô hướng có 5 đỉnh và 7 cạnh.
Gợi ý: Tổng số bậc = $2E = 2 \times 7 = 14$.

Tóm kết Chương 12

Hiểu rõ 2 cách biểu diễn Ma trận kề và Danh sách kề là nền tảng bắt buộc để cài đặt các thuật toán BFS, DFS và Dijkstra.

PART XIII

DUYỆT ĐỒ THỊ (BFS & DFS TRAVERSALS)

1. Intuition (Trực giác & Động lực)

Duyệt đồ thị là quá trình ghé thăm hệ thống tất cả các đỉnh của đồ thị. **BFS (Chiều rộng)** loang dần theo từng lớp khoảng cách từ đỉnh xuất phát (dùng **Queue**). **DFS (Chiều sâu)** đi sâu nhất có thể theo một nhánh trước khi quay lùi (Backtrack) (dùng **Stack** hoặc Đệ quy).

2. Formal Model & Algorithms

- BFS Algorithm:** Khởi tạo `queue Q`, đánh dấu `visited[start] = true`. Lặp khi Q không rỗng: lấy `u = Q.front()`, thăm `u`, với mỗi `v` kề `u` chưa ghé thăm: đánh dấu `visited[v] = true` và `Q.push(v)`.
- DFS Algorithm:** Đánh dấu `visited[u] = true`. Với mỗi `v` kề `u` chưa ghé thăm: gọi đệ quy `DFS(v)`.
- Xử lý đồ thị không liên thông:** Cần một vòng lặp ngoài duyệt qua toàn bộ V đỉnh. Nếu `!visited[i]` thì gọi `BFS(i)` hoặc `DFS(i)`.

3. Visual Diagram (Mô tả Loang BFS vs Đi sâu DFS)

BFS (Chiều Rộng)

Sử dụng QUEUE (FIFO)

DFS (Chiều Sâu)

Sử dụng STACK (LIFO)

4. C++ Implementation (BFS & DFS)

```
#include <vector>
#include <queue>
#include <iostream>
#include <functional>
#include <utility>

void BFS(int start, const std::vector<std::vector<int>>& adj, int V) {
    std::vector<bool> visited(V, false);
```

```
std::queue<int> q;
visited[start] = true; q.push(start);

while (!q.empty()) {
    int u = q.front(); q.pop();
    std::cout << u << " ";
    for (int v : adj[u]) {
        if (!visited[v]) {
            visited[v] = true;
            q.push(v);
        }
    }
}

void DFSUtil(int u, const std::vector<std::vector<int>>& adj, std::vector<bool>& visited) {
    visited[u] = true;
    std::cout << u << " ";
    for (int v : adj[u]) {
        if (!visited[v]) DFSUtil(v, adj, visited);
    }
}
```

5. Complexity Analysis

Cả BFS và DFS biểu diễn bằng Danh sách kề đều đạt độ phức tạp thời gian $\Theta(V + E)$ và bộ nhớ phụ $\Theta(V)$ cho mảng `visited` và Queue/Stack.

6. Dry Run Table (Trace BFS Từ Đỉnh 0)

Đồ thị: $adj[0]=\{1, 2\}$, $adj[1]=\{0, 3\}$, $adj[2]=\{0\}$, $adj[3]=\{1\}$:

Bước	Đỉnh u đang xét	Hành động Hàng đợi Queue	Thứ tự ghé thăm (Visited)
Khởi tạo	-	Push(0) $\Rightarrow$ Q = [0]	0
1	Pop 0	Push con kề chưa thăm: Push(1), Push(2) $\Rightarrow$ Q = [1, 2]	0, 1, 2
2	Pop 1	Push con kề chưa thăm: Push(3) $\Rightarrow$ Q = [2, 3]	0, 1, 2, 3
3	Pop 2	Không có con chưa thăm $\Rightarrow$ Q = [3]	0, 1, 2, 3
4	Pop 3	Queue rỗng $\Rightarrow$ Kết thúc BFS	0, 1, 2, 3

7. Common Errors & IT003 Exam Style

IT003 EXAM STYLE

Bẫy Đề Thi 1: Thứ tự ghé thăm đỉnh kề phụ thuộc vào cấu trúc dữ liệu! Nếu dùng Ma trận kề, các đỉnh kề thường được duyệt theo thứ tự chỉ số tăng dần. Đọc kỹ yêu cầu đề bài về thứ tự ưu tiên (VD: "ưu tiên đỉnh có số thứ tự nhỏ hơn").

Bẫy Đề Thi 2: Trong BFS, phải đánh dấu `visited[v] = true` ngay khi push `v` vào Queue, không được chờ tới khi pop `v` ra mới đánh dấu! Đánh dấu trễ sẽ khiến đỉnh `v` bị push lặp lại nhiều lần vào Queue.

8. Quick Recall & Exercise Ladder

- Level 0:** Thuật toán nào giúp tìm đường đi ngắn nhất (số cạnh ít nhất) giữa 2 đỉnh trên đồ thị không trọng số?
Đáp án: BFS (Breadth-First Search).
- Level 1:** Cho đồ thị có 4 đỉnh 0, 1, 2, 3 với các cạnh (0-1), (0-2), (1-3). Viết thứ tự duyệt DFS từ đỉnh 0 (ưu tiên đỉnh nhỏ hơn).
Gợi ý: $0 \Rightarrow 1 \Rightarrow 3 \Rightarrow 2$.

Tóm kết Chương 13

BFS dùng Queue loang theo tầng, DFS dùng Stack/Đệ quy đi sâu. Nhớ kỹ vị trí đánh dấu `visited` trong BFS để tránh lặp đỉnh!

PART XIV

THUẬT TOÁN ĐƯỜNG ĐI NGẮN NHẤT (SHORTEST PATH - DIJKSTRA)

1. Intuition (Trực giác & Động lực)

Bài toán tìm đường đi ngắn nhất giữa 2 địa điểm trên bản đồ Google Maps là ứng dụng thực tế phổ biến nhất của thuật toán **Dijkstra**. Thuật toán hoạt động theo nguyên lý tham ăn (Greedy): luôn chọn đỉnh có khoảng cách tạm thời nhỏ nhất để cố định (Fix/Finalize) đường đi ngắn nhất tới đỉnh đó.

2. Formal Model & Relaxation Principle

Điều kiện tiên quyết: Đồ thị có **trọng số không âm** ($w(e) \geq 0$). Thao tác **Nới cạnh (Relaxation)** giữa cạnh (u, v) có trọng số w :

$$\text{If } d[u] + w < d[v] \Rightarrow d[v] = d[u] + w, \quad p[v] = u$$

3. Visual Diagram (Sơ đồ Nối Cạnh Relaxation)



4. C++ Implementation (Dijkstra using Priority Queue)

```
#include <vector>
#include <queue>
#include <iostream>
#include <functional>
#include <utility>

const int INF = 1e9;
using pii = std::pair<int, int>; // (d[u], u)

void dijkstra(int start, const std::vector<std::vector<pii>>& adj, int V) {
    std::vector<int> d(V, INF);
    std::priority_queue<pii, std::vector<pii>, std::greater<pii>> pq;

    d[start] = 0;
    pq.push({0, start});

    while (!pq.empty()) {
        auto [dist, u] = pq.top(); pq.pop();
        if (dist > d[u]) continue;

        for (auto& edge : adj[u]) {
            int v = edge.first, w = edge.second;
            if (d[u] + w < d[v]) {
                d[v] = d[u] + w;
                pq.push({d[v], v});
            }
        }
    }
}
```

5. Algorithms & Complexity Analysis Comparison

Bên cạnh Dijkstra, ta có các thuật toán khác để xử lý đồ thị có trọng số âm hoặc tìm đường đi ngắn nhất mọi cặp đỉnh:

- **Bellman-Ford:** Lặp nối lỏng toàn bộ E cạnh đúng $V - 1$ lần, rồi kiểm tra thêm một lượt để phát hiện chu trình âm có thể đi tới từ nguồn. Độ phức tạp: $\Theta(V \times E)$.

- **Floyd-Warshall (All-Pairs):** Dùng Quy hoạch động, $D[i][j] = \min(D[i][j], D[i][k] + D[k][j])$. Cho phép cạnh có trọng số âm, nhưng nếu đồ thị chứa chu trình âm (Negative Cycle) thì khái niệm đường đi ngắn nhất hữu hạn không còn hợp lệ. Độ phức tạp: $\Theta(V^3)$.

Thuật toán	Bài toán	Trọng số âm	Độ phức tạp (Thời gian)
Dijkstra (Mảng)	Một nguồn (Single-Source)	✗ Không hỗ trợ	$\Theta(V^2)$
Dijkstra (Min-Heap)	Một nguồn (Single-Source)	✗ Không hỗ trợ	$\Theta((V + E) \log V)$
Bellman-Ford	Một nguồn (Single-Source)	✓ Hỗ trợ (Bảo chu trình âm)	$\Theta(V \times E)$
Floyd-Warshall	Mọi cặp (All-Pairs)	✓ Hỗ trợ	$\Theta(V^3)$

6. Dry Run Table (Trace Dijkstra Chuẩn Đề Thi IT003)

Tìm đường đi từ A (đỉnh 0) đến các đỉnh 1, 2, 3 với các cạnh (0-1: 4), (0-2: 2), (2-1: 1), (1-3: 5):

Bước	Đỉnh u chọn cố định	d[0]	d[1]	d[2]	d[3]	Hành động Nối Cạnh
0	-	0	∞	∞	∞	Khởi tạo d[0]=0
1	0 (d[0]=0)	0	4 (từ 0)	2 (từ 0)	∞	Nối cạnh (0-1:4), (0-2:2)
2	2 (d[2]=2)	0	3 (từ 2)	2	∞	Nối cạnh (2-1: 2+1=3 < 4) $\Rightarrow$ Cập nhật d[1]=3!
3	1 (d[1]=3)	0	3	2	8 (từ 1)	Nối cạnh (1-3: 3+5=8) $\Rightarrow$ Cập nhật d[3]=8
4	3 (d[3]=8)	0	3	2	8	Tất cả các đỉnh đã cố định đường đi ngắn nhất!

7. Common Errors & IT003 Exam Style

IT003 EXAM STYLE

Bẫy Đề Thi: Thuật toán Dijkstra **KHÔNG** chạy đúng khi đồ thị có cạnh mang trọng số âm! Nếu đồ thị có trọng số âm, phải chuyển sang dùng thuật toán Bellman-Ford.

8. Quick Recall & Exercise Ladder

- **Level 0:** Thuật toán Dijkstra có làm việc được trên đồ thị có chu trình âm không?
Đáp án: Không. Chu trình âm khiến khoảng cách giảm vô tận thành $-\infty$.

- **Level 1:** Trình bày bảng trace Dijkstra tìm đường đi ngắn nhất từ đỉnh 0 với đồ thị có các trọng số cho trước.
Gợi ý: Lập bảng cột đỉnh, cố định đỉnh có $d[u]$ nhỏ nhất ở mỗi dòng.

Tóm kết Chương 14

Dijkstra chọn đỉnh tham ăn có $d[u]$ nhỏ nhất để cố định đường đi. Đảm bảo thuộc lòng các bước lập bảng nối cạnh để ghi trọn điểm câu đồ thị cuối kỳ!

PART XV

EXAM TOOLKIT & BỘ ĐỀ ÔN THI MẪU CHUẨN IT003

1. Quick Recall Cheat Sheet (Tóm Tắt Nhanh Ôn Thi)

Master Summary Cheat Sheet

- **Selection & Interchange Sort:** Phép so sánh luôn cố định $C = \frac{n(n-1)}{2}$. Interchange Sort là thuật toán đặc thù UIT.
- **Binary Search:** Bắt buộc mảng đã sắp xếp. $\Theta(\log n)$ thời gian.
- **BST Deletion (2 con):** Đọc kỹ đề dùng Max-Left hay Min-Right làm phần tử thế mạng.
- **AVL Rotation:** $BF = h_{left} - h_{right}$. $+2$ nghiêng trái (LL/LR), -2 nghiêng phải (RR/RL).
- **B-Tree Bậc 5 ($m = 5$):** Max 4 keys, Min 2 keys per node. Tràn 5 keys $\Rightarrow$ Tách nút đẩy nổi khóa thứ 3 (trung vị) lên cha.
- **Hash Probing Unsuccessful Search:** Phải dò tới khi gặp ô **EMPTY** đầu tiên. Ô **DELETED** không dừng!
- **BFS / DFS:** BFS dùng Queue loang theo lớp; DFS dùng Stack/Đệ quy đi sâu. Tùy thuộc vào thứ tự đỉnh kề.
- **Dijkstra:** Chỉ áp dụng cho trọng số không âm ($w \geq 0$). Nối cạnh: `if (d[u] + w < d[v]) d[v] = d[u] + w`.

2. Phương pháp Ôn thi IT003 Hiệu quả

Một tỷ lệ đáng kể câu hỏi trong các đề thi IT003 đã phân tích yêu cầu sinh viên phải **chạy vết (trace) thủ công**, vẽ lại trạng thái cấu trúc dữ liệu sau mỗi bước, hoặc suy luận độ phức tạp dựa trên convention đặc thù. Học thuộc lòng code mà không hiểu cơ chế chạy tay sẽ mất điểm phần lớn các câu tự luận.

3. Integrated Practice Bank (Hệ thống Bài tập 5 Level)

Chủ đề 1: Sorting Algorithms (Thuật toán Sắp xếp)

- **Level 0 (Concept):** Thuật toán sắp xếp nào có Best Case là $\Theta(n)$ khi mảng đã có thứ tự? (Đáp án: Bubble Sort tối ưu, Insertion Sort).
- **Level 1 (Mini Trace):** Cho mảng $A = [8, 4, 2]$. Chạy tay 1 lượt thuật toán Bubble Sort. (Đáp án: $[4, 2, 8]$).

- **Level 2 (Full Trace):** Chạy tay toàn bộ thuật toán Selection Sort trên mảng $[29, 10, 14, 37, 14]$. Xác định tính ổn định.
- **Level 3 (Code):** Viết hàm `void QuickSort(int a[], int left, int right)` dùng phân hoạch Lomuto.
- **Level 4 (Exam Style):** Tính số phép so sánh và số phép gán của Interchange Sort trên mảng $[5, 4, 3, 2, 1]$.
- **Level 5 (Challenge):** Cải tiến Bubble Sort để dừng sớm nếu không có bất kỳ lần đổi chỗ nào xảy ra trong 1 lượt. Viết code C++ và chứng minh Best Case $\Theta(n)$.

Chủ đề 2: Cây (BST, AVL, B-Tree, Heap)

- **Level 0 (Concept):** Chiều cao tối đa của cây AVL chứa N nút được chặn trên bởi hàm gì? (Đáp án: $\mathcal{O}(\log N)$).
- **Level 1 (Mini Trace):** Cho Max-Heap dạng mảng $[50, 30, 40, 10, 20]$. Nút lá bắt đầu từ index nào?
- **Level 2 (Full Trace):** Lần lượt chèn 10, 20, 30, 40, 50 vào B-Tree bậc 5 rỗng. Vẽ cây ở bước tách nút.
- **Level 3 (Code):** Viết hàm tìm node có giá trị lớn nhất (Max) trong BST.
- **Level 4 (Exam Style):** Chèn lần lượt 30, 20, 40, 10, 5 vào cây AVL. Trình bày chi tiết hệ số cân bằng (BF) tại bước gây mất cân bằng và vẽ cây sau khi Xoay LL.
- **Level 5 (Challenge):** Cho cây BST bị mất 1 nhánh, dùng mảng duyệt LNR (In-order) và NLR (Pre-order) để phục hồi lại chính xác hình dáng cây BST ban đầu.

Chủ đề 3: Hash Table (Bảng Băm)

- **Level 0 (Concept):** Linear Probing gặp hiện tượng gì làm giảm hiệu suất? (Đáp án: Primary Clustering - Cụm sơ cấp).
- **Level 1 (Mini Trace):** Hàm băm $h(k) = k \bmod 7$. Khóa 15 bị đụng độ tại ô 1. Dùng Quadratic Probing i^2 , lần dò tiếp theo là ô nào? (Đáp án: $(1 + 1^2) \bmod 7 = 2$).
- **Level 2 (Full Trace):** Băm các khóa 43, 23, 1, 0, 15, 31 vào bảng băm $M = 10$, dùng Double Hashing với $h_1(k) = k \bmod 10$, $h_2(k) = 7 - (k \bmod 7)$. Vẽ bảng băm cuối cùng.
- **Level 3 (Code):** Khai báo cấu trúc dữ liệu cho Bảng băm dùng Phương pháp Nối kết trực tiếp (Separate Chaining).
- **Level 4 (Exam Style):** Tính số phép so sánh trung bình tìm kiếm thành công (C_{succ}) và không thành công (C_{unsucc}) cho Bảng băm Linear Probing kích thước 7 chứa 4 phần tử.
- **Level 5 (Challenge):** Cài đặt hàm Delete cho Linear Probing sử dụng cờ `DELETED` (Tombstone) để không làm đứt mạch dò tìm.

Chủ đề 4: Graph & Shortest Path (Đồ thị & Đường đi)

- **Level 0 (Concept):** Đồ thị vô hướng có V đỉnh và E cạnh. Tổng bậc của tất cả các đỉnh bằng bao nhiêu? (Đáp án: $2E$ - Định lý Bắt tay).
- **Level 1 (Mini Trace):** Viết thứ tự duyệt BFS bắt đầu từ đỉnh 0, biết đỉnh 0 kề với $\{1, 3, 2\}$ (duyet theo thứ tự đỉnh tăng dần). (Đáp án: 0, 1, 2, 3).
- **Level 2 (Full Trace):** Chạy tay thuật toán Dijkstra tìm đường đi ngắn nhất từ đỉnh A đến tất cả các đỉnh còn lại trên một đồ thị 5 đỉnh cho trước.
- **Level 3 (Code):** Viết hàm tính Bậc vào (In-degree) của một đỉnh v trên Đồ thị có hướng biểu diễn bằng Ma trận kề.
- **Level 4 (Exam Style):** So sánh độ phức tạp thời gian và điều kiện áp dụng của Dijkstra, Bellman-Ford, và Floyd-Warshall.
- **Level 5 (Challenge):** Phát hiện chu trình âm (Negative Cycle) trong đồ thị bằng cách chạy thêm 1 vòng lặp relaxation thứ V trong thuật toán Bellman-Ford.

4. Exam Simulator (Thi Thử Cuối Kỳ)

Câu 1 (2.5 điểm): Sắp Xếp & Đếm Phép Toán

Cho dãy số $A = [25, 12, 40, 8, 30]$.

a) Trình bày trạng thái mảng sau từng bước của Interchange Sort. Tính số phép so sánh C và số phép đổi chỗ M .

b) Thực hiện phân hoạch Lomuto đầu tiên cho Quick Sort với Pivot $a[high] = 30$.

Lời giải tóm tắt:

a) $C = 5(4)/2 = 10$ phép so sánh. Trạng thái sau từng i:

- $i = 0$: $\text{Swap}(25, 12) \Rightarrow \text{Swap}(12, 8) \Rightarrow [8, 25, 40, 12, 30]$
- $i = 1$: $\text{Swap}(25, 12) \Rightarrow [8, 12, 40, 25, 30]$
- $i = 2$: $\text{Swap}(40, 25) \Rightarrow [8, 12, 25, 40, 30]$
- $i = 3$: $\text{Swap}(40, 30) \Rightarrow [8, 12, 25, 30, 40]$. Mảng hoàn chỉnh! Total Swaps $M = 5$.

Câu 2 (2.5 điểm): Cây AVL

Lần lượt chèn các khóa sau vào cây AVL rỗng: 30, 20, 40, 10, 5. Xác định nút mất cân bằng và vẽ cây AVL sau mỗi lần xoay.

Lời giải tóm tắt:

- Chèn 30, 20, 40 $\Rightarrow$ Cây cân bằng hoàn hảo ($BF(30) = 0$).
- Chèn 10 $\Rightarrow$ Vẫn cân bằng ($BF(30) = +1, BF(20) = +1$).
- Chèn 5 $\Rightarrow$ Mất cân bằng LL tại nút 20 ($BF(20) = +2, BF(10) = +1$).
- Xoay Phải (Rotate Right) tại nút 20 $\Rightarrow$ Nút 10 lên làm cha của 5 và 20.
- Cây AVL hoàn chỉnh sau xoay: Gốc 30, Cây con trái là 10 (con trái 5, con phải 20), Cây con phải là 40 ($BF(30) = +1$, tất cả các nút $|BF| \leq 1$).

Câu 3 (2.5 điểm): Bảng Băm & B-Tree

Cho $M = 7, h(k) = k \bmod 7$. Chèn các khóa 14, 21, 7, 15 dùng Linear Probing. Tính C_{succ} trung bình.

Lời giải tóm tắt:

- $14 \% 7 = 0 \Rightarrow \hat{O} 0$ (1 probe)
 - $21 \% 7 = 0 \Rightarrow \hat{O} 1$ (2 probes)
 - $7 \% 7 = 0 \Rightarrow \hat{O} 2$ (3 probes)
 - $15 \% 7 = 1 \Rightarrow$ Đụng ô 1 $\Rightarrow \hat{O} 3$ (3 probes)
- $\Rightarrow C_{\text{succ}} = \frac{1+2+3+3}{4} = \frac{9}{4} = 2.25$ phép so sánh.

Câu 4 (2.5 điểm): Đồ thị Dijkstra

Lập bảng trace Dijkstra tìm đường đi ngắn nhất từ đỉnh 0 cho đồ thị 4 đỉnh với ma trận trọng số cho trước.

Lời Chúc Thi Tốt từ Ban Biên Soạn

Chúc các bạn sinh viên UIT ôn luyện hiệu quả, tự tin áp dụng tư duy chạy tay và đạt điểm 10 tuyệt đối trong kỳ thi môn IT003!